

Internal verification of assessment decisions - BTEC Higher Nationals (RQF)

INTERNAL VERIFICATION - ASSESSMENT DECISIONS (Single Student)					
Programme Title:		HND in Computing			
Unit Number and Title:		20. Applied Programming and Design Principles			
Assessor Name:		Ms. ALF. Sajeetha	Internal Verifier Name:		
Assignment title:		Delta Transit Authority (DTA) Passenger Analytical Tool			
Name of student	Submission Type (First, Resubmission, Retake)	Grade the Assessor has awarded.	Assessment Decision Accurate (Y/N) (Resubmission and retake must be capped at a Pass)	List the learning outcomes and grading criteria where inaccurate decisions have been made	State why the assessment decision is inaccurate. <i>*If an inaccurate decision is recorded the Internal Verifier must recommend actions detailing the issues to be addressed. The Assessor and the Internal Verifier must then confirm that the action has been undertaken before assessment decisions are issued to the student.</i>
INTERNAL VERIFIER CHECKLIST					Y/N
Has the student and the Assessor confirmed the authenticity of the evidence?					
Is there evidence of collusion or plagiarism? The work submitted for assessment has been carried out without assistance other than that which is acceptable according to the rules of the specification. The evidence submitted for this assignment is student's own. The student has clearly referenced any sources and any artificial intelligence (AI) tools used in the work.					
Is the assessor feedback to the student appropriate and constructive to each student? Provide comments below. • Points out strengths and areas for improvement. • Linked to relevant learning outcomes and assessment criteria. • Clear as to why the student did not achieve higher grades.					

- Identify opportunities for improved performance in future assignments.

GENERAL COMMENTS

Any actions required must be reviewed across the whole cohort.

Action Required	Target Date for Completion	Date Action Completed

I confirm that the assessment decisions are accurate, there is no evidence of assessment malpractice and any action points have been addressed and completed in respect of the whole cohort. I declare the work submitted for assessment has been carried out without assistance other than that which is acceptable according to the rules of the specification. The evidence submitted for this assignment is student's own. The student has clearly referenced any sources and any artificial intelligence (AI) tools used in the work.

Internal Verifier signature		Date	
Assessor signature		Date	

Higher Nationals - Summative Assignment Feedback Form

Student Name/ID	MN. Sihanas / 1030371		
Unit Title	Applied Programming and Design Principles		
Assignment	Assignment 01	Assessor	Ms. ALF. Sajeetha
Submission Date	26.08.2025	Date Received 1st	
Re-submission Date		Date Received 2nd submission	
Assessor Feedback: *Please note that constructive and useful feedback should allow students to understand: a) Strengths of performance b) Limitations of performance c) Any improvements needed in future assessments Feedback should be against the learning outcomes and assessment criteria to help students understand how these inform the process of judging the overall grade. Feedback should give full guidance to the students on how they have met the learning outcomes and assessment criteria.			
<i>I certify that to the best of my knowledge the evidence submitted for this assignment is the student's own. The student has clearly referenced any sources and any artificial intelligence (AI) tools used in the work. I have not solely used AI to grade the student's work.</i>			
Grade:	Assessor Signature:		Date:
Resubmission Feedback: *Please note resubmission feedback is focussed only on the resubmitted work			
Grade:	Assessor Signature:		Date:
Internal Verifier's Comments:			
Signature & Date:			

* Please note that grade decisions are provisional. They are only confirmed once internal and external moderation has taken place and grades decisions have been agreed at the assessment board.

STUDENT ASSESSMENT SUBMISSION AND DECLARATION

When submitting evidence for assessment, each student must sign a declaration confirming that the work is their own.

Student name: MN. Sihan		Assessor name: Ms. ALF. Sajeetha	
Issue date: 26.06.2025	Submission date: 31.08.2025		Submitted on: 31.08.2025
Programme: HND in Computing			
Unit: Programming			
Assignment number and title: Delta Transit Authority (DTA) Passenger Analytical Tool			

Plagiarism

Plagiarism is a particular form of cheating. Plagiarism must be avoided at all costs and students who break the rules, however innocently, may be penalised. It is your responsibility to ensure that you understand correct referencing practices. As a university level student, you are expected to use appropriate references throughout and keep carefully detailed notes of all your sources of materials for material you have used in your work, including any material downloaded from the Internet. Please consult the relevant unit lecturer or your course tutor if you need any further advice.

Student Declaration

Student declaration

I certify that the assignment submission is entirely my own work and I fully understand the consequences of plagiarism. I declare that the work submitted for assessment has been carried out without assistance other than that which is acceptable according to the rules of the specification. I certify I have clearly referenced any sources and any artificial intelligence (AI) tools used in the work. I understand that making a false declaration is a form of malpractice.

Student signature: 1030371@bcas.ac

Date: 26.08.2025

General Guidelines

1. A Cover page or title page - You should always attach a title page to your assignment. Use previous page as your cover sheet and make sure all the details are accurately filled.
2. Attach this brief as the first section of your assignment.
3. All the assignments should be prepared using a word processing software.
4. All the assignments should be printed on A4 sized papers. Use single side printing.
5. Allow 1" for top, bottom, right margins and 1.25" for the left margin of each page.

Word Processing Rules

1. The font size should be 12 point, and should be in the style of Time New Roman.
2. Use 1.5 line spacing. Left justify all paragraphs.
3. Ensure that all the headings are consistent in terms of the font size and font style.
4. Use footer function in the word processor to insert Your Name, Subject, Assignment No, and Page Number on each page. This is useful if individual sheets become detached for any reason.
5. Use word processing application spell check and grammar check function to help editing your assignment.

Important Points

1. It is strictly prohibited to use text boxes to add texts in the assignments, except for the compulsory information. eg: Figures, tables of comparison etc. Adding text boxes in the body except for the before mentioned compulsory information will result in rejection of your work.
2. Avoid using page borders in your assignment body.
3. Carefully check the hand in date and the instructions given in the assignment. Late submissions will not be accepted.
4. Ensure that you give yourself enough time to complete the assignment by the due date.
5. Excuses of any nature will not be accepted for failure to hand in the work on time.
6. You must take responsibility for managing your own time effectively.
7. If you are unable to hand in your assignment on time and have valid reasons such as illness, you may apply (in writing) for an extension.
8. Failure to achieve at least PASS criteria will result in a REFERRAL grade.
9. Non-submission of work without valid reasons will lead to an automatic REFERRAL. You will then be asked to complete an alternative assignment.
10. If you use other people's work or ideas in your assignment, reference them properly using HARVARD referencing system to avoid plagiarism. You have to provide both in-text citation and a reference list.
11. If you are proven to be guilty of plagiarism or any academic misconduct, your grade could be reduced to A REFERRAL or at worst you could be expelled from the course.

Unit 20: Applied Programming and Design Principles

Assignment Brief

Programme Title	
Student Name/ID Number	MN. Sihanass / 1030371
Unit Number and Title	Unit 20: Applied Programming and Design Principles
Academic Year	
Unit Tutor	Ms. ALF. Sajeetha
Assignment Title	Delta Transit Authority (DTA) Passenger Analytical Tool
Issue Date	26.06.2025
Submission Date	26.08.2025
Submission Format	
The submission should be in the form of an individual report written in a concise, formal business style using single spacing and font size 12. You are required to make use of headings, paragraphs and subsections as appropriate, and all work must be supported with research and referenced using Harvard referencing system. Please also provide an end list of references using the Harvard referencing system. The recommended word count is 5,000–5,500 words for the report excluding annexures, although you will not be penalised for exceeding the total word limit.	
Unit Learning Outcomes	
L01 Investigate the impact of SOLID development principles on the OOP paradigm. L02 Design a large dataset processing application using SOLID principles and clean coding techniques L03 Build a data processing application based on a developed design. L04 Perform automatic testing on a data processing application.	

Transferable skills and competencies developed
Python Frameworks of Desktop GUI development, Automation Python Unit Testing Tool
Vocational scenario - Delta Transit Authority (DTA) Passenger Analytical tool
Delta Transit Authority (DTA) is a leading public transportation provider in a major Delta city, serving millions of passengers daily across subway, bus, and commuter rail systems. With a vast network of stations and routes, DTA collects massive amounts of data from turnstiles, mobile ticketing apps, and payment systems. Currently, the DTA uses a computerized system to record daily passenger entries and exits, route selections, and ticket purchases, storing all data in a centralized database. Every quarter, the DTA management team analyzes this passenger data to make decisions about service planning, resource allocation, and infrastructure upgrades. However, the current data analysis process is manual, leading to several challenges such as human bias, limited analytical scope, slow turnaround, high error rates, increased costs, and poor reproducibility. Recognizing these issues, DTA's top management has decided to commission a computer-based solution for passenger data analysis. You are a lead software engineer at Urban Data Solutions (UDS) Ltd, a leading software development firm specializing in large-scale analytics. DTA's management has contracted UDS to develop a standalone desktop application for passenger data analysis. The application must be built using Python Framework and should provide a GUI for user interaction. Several large, real-world sample datasets are available at the following sources, which you should use for development and testing. 1. National Transit Database (NTD) (FTA) https://www.transit.dot.gov/ntd 2. MTA Ridership and Transit Data (Data.gov) https://catalog.data.gov/dataset/?tags=passengers , https://catalog.data.gov/dataset/?tags=passengers 3. Public Transportation Passenger Counts (Kaggle) https://www.kaggle.com/datasets/ifuurh/public-transportation-passenger-counts

Using above sources, implement chunk-based processing for large files (5GB+) including robust error handling and a dead-letter queue for problematic records and build a modular, extensible pipeline using object-oriented programming principles, including,

1. Source-agnostic data ingestion (support for CSV, JSON, Database or APIs)
2. Chainable data transformations (filtering, aggregation, enrichment)
3. Multi-format output (Graph, Forms or CSV)

The DTA management expects the following results from your computer-based passenger data analysis application,

1. **Peak Congestion Analysis:**
Passenger data should help identify the times and stations with the highest congestion. This information will be used to optimize train and bus schedules, allocate resources, and improve passenger flow. (Analysis by hour, day, month, and year.)
2. **Popular Route and transfer point Analysis:**
The application should analyze passenger flow to determine the most frequently used routes and transfer points. This will inform service planning and marketing strategies.
3. **Service Disruption Detection:**
By analyzing anomalies in passenger flow, the system should help detect potential service disruptions or unusual patterns that may indicate operational issues.
4. **Regional Performance Analysis:**
The application should provide insights into passenger trends across different regions or boroughs, enabling targeted improvements and investments.

Assignment activity and guidance

PART 1

By evaluating the impact of **SOLID** development principles on object-oriented

application development. You should consider the following areas in preparing that report,

Investigation about Principles of **object-oriented paradigm, Class relationships** with coding examples.

Analysis in different types of **design patterns** with coding examples.

Explanation about **clean coding techniques** and the impact of clean coding techniques on the use of data structures and their operations in writing algorithms.

Evaluation about the impact of **SOLID** development principles on object-oriented application development using appropriate coding examples.

Part 2

Prepare a design document to design a large dataset processing application (Low level Structural Design of the Application using UML) for the given scenario. You should consider the following areas in preparing that document,

Explanation about utilization of solid principles, clean coding techniques and design patterns in the proposed application.

Explanation about how the design of the proposed application is refined to include multiple design patterns.

Design of the test plan (testing regime) for the proposed application by including a provision for automated testing.

[Test Plan Template](#)

Part 3

Build (develop) the proposed data processing application using Python programming language. Prepare a report by providing the screenshots of the source codes and user interfaces as **evidence** and **assessing** the effectiveness of using SOLID principles, clean coding techniques and programming design patterns on the application developed. (Attach the source code)

Part 4

Perform automated testing on the developed application. You are required to prepare a report about that automated testing by considering following areas,

Examination about the different methods of implementing automatic testing as designed in the test plan.

Screenshots of the implemented automated testing as **evidence**.

Discussion about the differences between developer-produced and vendor-provided automatic testing tools for applications and software systems.

Analysis about the benefits and drawbacks of different forms of automatic testing of applications and software systems, with examples from your developed data processing application. (Ex: Binary searching algorithm python coding – unit testing)

Recommended Resources

Please note that the resources listed are examples for you to use as a starting point in your research – the list is not definitive.

Learning Outcomes and Assessment Criteria

Pass	Merit	Distinction
LO1 Investigate the impact of SOLID development principles on the OOP paradigm		D1 Evaluate the impact of SOLID development principles on object orientated application development.
P1 Investigate the characteristics of the object orientated paradigm, including class relationships and SOLID principles.	M1 Analyse, with examples, each of the creational, structural and behavioural design pattern types.	
P2 Explain how clean coding techniques can impact on the use of data structures and operations when writing algorithms.		
LO2 Design a large dataset processing application using SOLID principles and clean coding techniques		
P3 Design a large data set processing application, utilising SOLID principles, clean coding techniques and a design pattern.	M2 Refine the design to include multiple design patterns	

P4 Design a suitable testing regime for the application, including provision for automated testing.		
LO3 Build a data processing application based on a developed design		D2 Analyse the benefits and drawbacks of different forms of automatic testing of applications and software systems, with examples from the developed application
P5 Build a large dataset processing application based on the design produced	M3 Assess the effectiveness of using SOLID principles, clean coding techniques and programming patterns on the application developed.	
LO4 Perform automatic testing on a data processing application		
P6 Examine the different methods of implementing automatic testing as designed in the test plan. P7 Implement automatic testing of the developed application.	M4 Discuss the differences between developer produced and vendor provided automatic testing tools for applications and software systems.	

Acknowledgement

I would like to express my sincere gratitude to all those who supported me throughout the completion of this assignment. First and foremost, I extend my deepest appreciation to my assessor, Mrs. ALF. Sajeetha, for her valuable guidance, encouragement, and constructive feedback, which were instrumental in shaping this work.

I am also thankful to my lecturers and the faculty of BCAS Campus, whose continuous teaching and support provided me with the knowledge and resources required to carry out this project successfully.

A special thanks goes to my peers and colleagues who contributed with their feedback during the design and evaluation stages of the user interface, enabling me to refine the system based on real user perspectives. Their insights were crucial in making this project more practical and user-centered.

Finally, I would like to acknowledge my family for their patience, understanding, and constant encouragement, which motivated me to stay focused and complete this work to the best of my ability.

Table of Content

Introduction.....	1
Part 1	2
1.1 Investigation about Principles of Object-Oriented Paradigm and Class Relationships	2
1.1.1 Principles of Object-Oriented Paradigm.....	2
1.1.2 Class Relationships in OOP	6
1.2 Analysis of Different Types of Design Patterns with Coding Examples	9
1.2.1 Creational Design Patterns.....	9
1.2.2 Structural Design Patterns	11
1.2.3 Behavioural Design Patterns.....	14
1.3 The impact of clean coding techniques on the use of data structures and operations when writing algorithms	16
1. Meaningful Naming.....	16
1.4 Evaluation on the Impact of SOLID Principles in Object-Oriented Application Development	23
1.4.1. Single Responsibility Principle (SRP).....	23
1.4.2. Open–Closed Principle (OCP).....	24
1.4.3. Liskov Substitution Principle (LSP)	25
1.4.4. Interface Segregation Principle (ISP)	26
1.4.5. Dependency Inversion Principle (DIP)	27
Part 2	28
2.1 Utilization of SOLID Principles, Clean Coding Techniques, and Design Patterns in the Proposed Application ...	28
2.1.1 Utilization of SOLID Principles	28
2.1.2 Utilization of Clean Coding Techniques.....	30
3.1.3 Utilization of Design Patterns.....	31
2.2 Explanation about how the design of the proposed application is refined to include multiple design patterns.	32
2.3 Designing a test plan (testing regime) for the proposed application by including a provision for automated testing.	35
2.3.1 Testing Strategies.....	35
2.3.2 Test Plan for the Proposed Application	35
Part 3	38
3.1 Source Code of the Developed System with evidence.....	38
3.1.1 Core Abstractions	38
3.1.2 Data Access & Preprocessing	40
3.1.3 Search & Filtering Orchestration	41
3.1.4 Analytical Modules.....	42
3.1.5 Application Orchestration.....	50
1.3.6 Entry Point.....	58
3.2 Interface overview of the Developed System with evidence.	59
3.2.1 Sidebar Filters.....	59
3.2.2 Overview Metrics Section	60
3.2.6 Regional Performance analysis Tab.....	65
3.3 Assessment of SOLID Principles, Clean Code Practices, and Design Patterns.....	66
3.3.1 Application of SOLID Principles	66

3.3.2 Clean Code Practices	67
3.3.3 Use of Design Patterns.....	68
Part 4	69
4.1 Examination of Automated Testing Methods	69
4.1.1 Unit Testing:	69
4.1.2 Integration Testing:.....	71
4.1.3 Mocking and Stubbing:.....	72
4.1.4 Error Handling Tests:	73
4.1.5 Coverage Measurement:	73
4.2 Evidence of Implemented Automated Testing.....	74
4.2.1 Pytest Execution Results:.....	74
4.2.2 Coverage Report:	75
4.3 Discussion on Developer-Produced vs Vendor-Provided Automatic Testing Tools	77
4.3.1 Developer-Produced Testing Tools	77
4.3.2 Vendor-Provided Testing Tools.....	78
4.3.3 Comparative Analysis.....	79
4.4 Analysis of the Benefits and Drawbacks of Different Forms of Automatic Testing	80
4.4.2 Integration Testing.....	81
4.4.3 Mocking and Stubbing.....	82
4.4.4 Coverage Testing	82
4.4.5 Comparative Summary of Benefits and Drawbacks of Different Forms of Automatic Testing	83
Conclusion	84
References.....	85

List of Figure

Figure 1: coding example for Abstraction	3
Figure 2: Coding example for Encapsulation	4
Figure 3: Coding example for Inheritance	4
Figure 4: Coding example for Polymorphism	5
Figure 5: Coding example for Association class relationship	6
Figure 6: Coding example for Aggregation class relationship	7
Figure 7: Coding example for Composition class relationship	7
Figure 8: Coding example for Dependency class relationship	8
Figure 9: Coding example for Generalization (Inheritance) class relationship	8
Figure 10: Coding example for singleton method design pattern	9
Figure 11: Coding example for factory method design pattern	10
Figure 12: Coding example for Adapter method Design pattern	11
Figure 13: Coding example for Façade method design pattern	12
Figure 14: Coding example for Strategy method design pattern	14
Figure 15: Coding example for Template method design pattern	15
Figure 16: Before using Meaningful Naming technique	16
Figure 17: after using Meaningful naming technique	16
Figure 18: Before using Single Responsibility Principle (SRP)	17
Figure 19: after using Single Responsibility Principle (SRP)	17
Figure 20: before using Consistent Formatting and Code Structure	18
Figure 21: after using Consistent Formatting and Code Structure	18
Figure 22: before using Modularity and Reusability	19
Figure 23: after using Modularity and Reusability	19
Figure 24: before using Error Handling and Robustness	19
Figure 25: after using Error Handling and Robustness	20
Figure 26: before using Avoiding Code Duplication	20
Figure 27: after using Avoiding Code Duplication	21
Figure 28: before using Readability and Documentation	21
Figure 29: after using Readability and Documentation	22
Figure 30: Coding example for Single Responsibility Principle (SRP)	23
Figure 31: Coding example for Open–Closed Principle (OCP)	24
Figure 32: Coding example for Liskov Substitution Principle (LSP)	25
Figure 33: Coding example for Interface Segregation Principle (ISP)	26
Figure 34: Coding example for Dependency Inversion Principle (DIP)	27
Figure 35: UML class diagram before refinements	28
Figure 36: UML class diagram after the refinements	34
Figure 37: DataAnalyzer class	38
Figure 38: DataFilter class	38
Figure 39: TimeFilter Class	39
Figure 40: RegionFilter class	39
Figure 41: RouteFilter class	39
Figure 42: DataProcessor class	40
Figure 43: SearchEngine class	41
Figure 44: CongestionAnalyzer class 1	42
Figure 45: CongestionAnalyzer class 2	43
Figure 46: RouteAnalyzer class 1	44
Figure 47: RouteAnalyzer class 2	45
Figure 48: DisruptionAnalyzer class 1	46
Figure 49: DisruptionAnalyzer class 2	47
Figure 50: RegionalAnalyzer class 1	48
Figure 51: RegionalAnalyzer class 2	49
Figure 52: BusPassengerAnalysisApp	50
Figure 53: load_data function	51
Figure 54: create_sidebar_filters function	51
Figure 55: apply_filters function	52
Figure 56: display_overview_metrics function	53
Figure 57: run function	54
Figure 58: run function tab 1	55

Figure 59: run function tab 2	55
Figure 60: run function tab 3	56
Figure 61: run function tab 4	57
Figure 62: run function tab 5	58
Figure 63: main function	58
Figure 64: sidebar (interface).....	59
Figure 65: Dashboard overview (interface).....	60
Figure 66: Peak Congestion Analysis tab (interface).....	61
Figure 67: Route analysis tab (interface) 1	62
Figure 68: Route analysis tab (interface) 2	63
Figure 69: Disruption Detection analysis Tab (interface) 1.....	64
Figure 70: Disruption Detection analysis Tab (interface) 2.....	64
Figure 71: Regional Performance analysis tab (interface).....	65
Figure 72: DataProcessor pytest	69
Figure 73: TimeFilter pytest	70
Figure 74: End-to-End Pipeline pytest.....	71
Figure 75: Mocking Streamlit Page Config pytest	72
Figure 76: File Not Found (error handling) pytest.....	73
Figure 77: coverage measurement pytest.....	73
Figure 78: pytest execution.....	74
Figure 79: pytest result (all passed)	74
Figure 80: function coverage report.....	75
Figure 81: class coverage report	76
Figure 82: unit testing example from the developed application	80
Figure 83: integration testing example from the developed application.....	81
Figure 84: Mocking and Stubbing example from the developed application	82
Figure 85: Comparative Summary of Benefits and Drawbacks of Different Forms of Automatic Testing.....	83

List of Table

Table 1: Testing Schedule 37

Table 2: Application of SOLID Principles 66

Table 3: Clean Code Practices 67

Table 4: Use of Design Patterns 68

Table 5: comparison of Developer-Produced vs Vendor-Provided Automatic Testing Tools 79

Introduction

The Delta Transit Authority (DTA) has a critical need for a modern, efficient passenger data analysis tool to replace its current manual system. This report details the development of a desktop application, the DTA Passenger Analytical Tool, built to address the challenges of human bias, limited scope, and high error rates associated with manual data processing. The solution is built upon a foundation of robust software engineering principles, with a particular focus on object-oriented programming (OOP), clean coding techniques, and the SOLID principles. The application is designed to handle large datasets, providing crucial insights into peak congestion, popular routes, service disruptions, and regional performance.

This document will investigate the theoretical underpinnings of the design, including an analysis of relevant design patterns and automatic testing methodologies, before concluding with a reflection on the effectiveness of the principles applied.

Part 1

1.1 Investigation about Principles of Object-Oriented Paradigm and Class Relationships

1.1.1 Principles of Object-Oriented Paradigm

Object-Oriented Programming (OOP) is a software development paradigm that models real-world entities as objects containing both data (attributes) and behaviours (methods). Instead of writing one long sequence of procedural code, OOP organizes programs into small, reusable, and modular components. This makes complex applications easier to design, maintain, and extend over time (Alexander S Gillis, 2024).

The **main advantages of OOP** include:

- Improved code reusability through inheritance and polymorphism.
- Better data security through encapsulation.
- Enhanced flexibility and maintainability because changes in one class do not always affect others.
- Ability to model real-world problems naturally.

The four key principles of OOP are Abstraction, Encapsulation, Inheritance, and Polymorphism.

1. Abstraction

Abstraction is the process of hiding implementation details and exposing only the necessary functionality. This allows developers to work with high-level concepts rather than being burdened with low-level details. Abstraction reduces complexity by defining a common interface that multiple classes can follow. This also improves flexibility since new implementations can be added without changing the client code (Pankaj, 2022).

Example: In the application, the DataAnalyzer abstract base class defines what analyzers should do (analyze and visualize data), but not how they should do it. Each analyzer (Congestion, Route, Disruption, Regional) provides its own implementation.

```

35 class DataAnalyzer(ABC):
36     """Abstract base class for all data analyzers - demonstrates abstraction"""
37
38     @abstractmethod
39     def analyze(self, data: pd.DataFrame) -> Dict:
40         """Abstract method that must be implemented by all analyzers"""
41         pass
42
43     @abstractmethod
44     def visualize(self, analysis_result: Dict) -> go.Figure:
45         """Abstract method for creating visualizations"""
46         pass

```

Figure 1: coding example for Abstraction

This ensures consistency across all analyzers while allowing each one to have different implementations. If tomorrow a new analyzer (e.g., “RevenueAnalyzer”) is added, it must follow the same interface without breaking the system.

2. Encapsulation

Encapsulation is the bundling of data (attributes) and methods (behaviours) into a single unit, while restricting direct access to the object’s internal state. Encapsulation protects data integrity by preventing unauthorized or accidental modification. It also provides controlled access through getters and setters (Sheldon, 2023).

Example: The DataProcessor class encapsulates the dataset. Raw data and processed data are stored in private attributes (`_data` and `_processed_data`) which cannot be accessed directly. Instead, the `data` property is used to safely retrieve it.

```

99 class DataProcessor:
100     """Handles all data loading and preprocessing operations"""
101
102     def __init__(self, file_path: str):
103         self._file_path = file_path
104         self._data = None
105         self._processed_data = None
106
107 > def load_data(self) -> pd.DataFrame: ...
116
117 > def _preprocess_data(self): ...
136
137 @property
138 def data(self) -> pd.DataFrame:
139     """Getter for processed data - demonstrates encapsulation"""
140     return self._processed_data if self._processed_data is not None else pd.DataFrame()

```

Figure 2: Coding example for Encapsulation

This ensures that other parts of the program cannot corrupt the raw dataset. Only valid, preprocessed data is made available, improving reliability and preventing runtime errors.

3. Inheritance

Inheritance allows one class (child/subclass) to acquire properties and methods from another class (parent/superclass). Inheritance promotes code reusability. Instead of writing the same code multiple times, common features are defined once in a parent class and reused across child classes (JANSSEN, 2017).

Example: The CongestionAnalyzer, RouteAnalyzer, DisruptionAnalyzer, and RegionalAnalyzer classes all inherit from the abstract base class DataAnalyzer. This ensures they all share the same structure but can implement their own logic.

```

146 class CongestionAnalyzer(DataAnalyzer):
147     """Analyzes peak congestion patterns - implements DataAnalyzer interface"""
148
149 > def analyze(self, data: pd.DataFrame) -> Dict: ...
171
172 > def visualize(self, analysis_result: Dict) -> Dict[str, go.Figure]: ...

```

Figure 3: Coding example for Inheritance

This eliminates duplication. All analyzers automatically follow the same pattern, which makes the system **extensible** (new analyzers can be added easily) and **consistent** (all analyzers have analyze and visualize methods).

4. Polymorphism

Polymorphism allows the same interface or method to take different forms depending on the object that is using it. It simplifies code because the client does not need to know which specific class is being used; it only interacts with the common interface (Janssen, 2021).

Example: Although every analyzer implements the same methods (analyze and visualize), the logic inside these methods is different. The system just calls `analyzer.analyze(data)` and gets the correct result depending on which analyzer is active.

```
758      # Polymorphic behavior: Different analyzers implement analyze() differently
759      analysis_result = analyzer.analyze(filtered_data)
760      figures = analyzer.visualize(analysis_result)
```

Figure 4: Coding example for Polymorphism

The application can dynamically switch between congestion, route, disruption, and regional analysis without changing the core logic. This makes the design flexible and future-proof.

1.1.2 Class Relationships in OOP

In object-oriented programming (OOP), the relationships between classes define how they interact and relate to each other. These connections are fundamental to building complex, well-structured, and maintainable software systems. Understanding these relationships helps in modelling real-world problems more accurately and creating code that is modular, reusable, and easy to manage.

The key class relationship types of OOP are association, aggregation, composition, dependency, and Generalization (Inheritance).

1. Association

Association is the most general and weakest relationship between two classes. It represents a "has a" or "uses a" connection where one class is related to another. The objects can exist independently of each other (Khan, 2023).

Example: BusPassengerAnalysisApp uses the SearchEngine class to perform filtering, but both can exist separately.

```
429 class BusPassengerAnalysisApp:
430     """Main application class implementing singleton pattern"""
431
432     _instance = None
433
434 > def __new__(cls): ...
438
439     def __init__(self):
440         if not hasattr(self, 'initialized'):
441             self.data_processor = None
442             self.search_engine = SearchEngine()
443 >             self.analyzers = { ...
449             self.initialized = True
```

Figure 5: Coding example for Association class relationship

It promotes modularity since the search functionality can be improved or replaced without affecting the entire app.

2. Aggregation

Aggregation is a specific type of association that represents a "whole-part" relationship, where the "part" can exist independently of the "whole." It's a weaker form of a "has a" relationship. A key characteristic is that if the "whole" object is destroyed, the "part" objects are not (Khan, 2023).

Example: BusPassengerAnalysisApp aggregates analyzers like CongestionAnalyzer, RouteAnalyzer, etc. These analyzers exist independently but are grouped under the main app.

```
443         self.analyzers = {
444             'congestion': CongestionAnalyzer(),
445             'route': RouteAnalyzer(),
446             'disruption': DisruptionAnalyzer(),
447             'regional': RegionalAnalyzer()
448         }
```

Figure 6: Coding example for Aggregation class relationship

3. Composition

Composition is a strong form of aggregation where the "part" objects cannot exist without the "whole" object. It's a "has a" relationship where the parts are dependent on the whole. When the "whole" object is destroyed, its "part" objects are also destroyed (Khan, 2023).

Example: The DataProcessor creates and manages _data and _processed_data. These objects are completely dependent on DataProcessor.

```
99     class DataProcessor:
100         """Handles all data loading and preprocessing operations"""
101
102         def __init__(self, file_path: str):
103             self._file_path = file_path
104             self._data = None
105             self._processed_data = None
```

Figure 7: Coding example for Composition class relationship

It guarantees integrity of the data pipeline. If DataProcessor is destroyed, its internal data is also destroyed.

4. Dependency

Dependency means one class relies on another to perform its function. Dependency is a "uses a" relationship where one class depends on another. It means that a change in one class might affect another, but the relationship is not a structural one like association or aggregation. This often occurs when one class uses another as a parameter in a method (sözer, 2023).

Example: All analyze methods depend on a pandas.DataFrame input. Without the dataset, the analyzer cannot function.

```
149     def analyze(self, data: pd.DataFrame) -> Dict:
150         """Analyze congestion patterns by different time periods"""
151         analysis = {}
```

Figure 8: Coding example for Dependency class relationship

This makes analyzers flexible because they work with any dataset as long as it is provided in the right format.

5. Generalization (Inheritance)

Generalization is the process of defining common features in a parent class that are shared by child classes. Generalization, more commonly known as Inheritance, represents an "is a" relationship. It allows a new class (the subclass or child class) to inherit properties and behaviours from an existing class (the superclass or parent class). This promotes code reuse and creates a hierarchical structure (sözer, 2023).

Example: The DataAnalyzer class generalizes all analyzer classes by defining the common interface (analyze, visualize).

```
35     class DataAnalyzer(ABC):
36         """Abstract base class for all data analyzers - demonstrates abstraction"""
37
38         @abstractmethod
39         def analyze(self, data: pd.DataFrame) -> Dict:
40             """Abstract method that must be implemented by all analyzers"""
41             pass
```

Figure 9: Coding example for Generalization (Inheritance) class relationship

This reduces duplication and enforces a standard design across analyzers.

1.2 Analysis of Different Types of Design Patterns with Coding Examples

Design patterns are proven solutions to common software design problems. They provide reusable templates that help developers build scalable, maintainable, and flexible applications. Design patterns are generally categorized into Creational, Structural, and Behavioral patterns:

- Creational Patterns → Deal with object creation mechanisms.
- Structural Patterns → Deal with class and object composition.
- Behavioral Patterns → Deal with communication and responsibility among objects.

1.2.1 Creational Design Patterns

Creational design patterns are concerned with how objects are created. They aim to separate the client from the details of object creation, providing more flexibility and control. Instead of directly instantiating objects with the `new` keyword, these patterns delegate the creation process to other objects or methods. This approach makes the system more flexible by allowing the object-creation mechanism to be modified independently of the classes that use them (GeeksforGeeks, 2023).

a) Singleton method design pattern

The Singleton pattern ensures that a class has only one instance and provides a global point of access to it. It's useful when you need to strictly control the number of objects created for a resource, such as a database connection pool or a configuration manager. A single instance helps manage shared resources and prevent conflicts (GeeksforGeeks, 2016).

```
class BusPassengerAnalysisApp:
    """Main application class implementing Singleton pattern"""

    _instance = None

    def __new__(cls):
        if cls._instance is None:
            cls._instance = super(BusPassengerAnalysisApp, cls).__new__(cls)
        return cls._instance
```

Figure 10: Coding example for singleton method design pattern

Code explanation: The BusPassengerAnalysisApp class implements the Singleton pattern to ensure only one instance of the application exists.

Impact: This prevents multiple dashboards from being initialized, ensuring data consistency and resource efficiency.

b) Factory method design pattern

The Factory Method pattern defines an interface for creating an object but lets subclasses decide which class to instantiate. It defers object instantiation to subclasses, making the code more flexible and scalable. This is particularly useful when a class can't anticipate the type of objects it needs to create (Gorai, 2024).

```
self.analyzers = {  
    'congestion': CongestionAnalyzer(),  
    'route': RouteAnalyzer(),  
    'disruption': DisruptionAnalyzer(),  
    'regional': RegionalAnalyzer()  
}
```

Figure 11: Coding example for factory method design pattern

Code explanation: The BusPassengerAnalysisApp acts like a factory for analyzers. Instead of creating them individually everywhere, it manages them in a single dictionary.

Impact: This makes the system extensible. If a new analyzer (e.g., RevenueAnalyzer) is added, it can be included in the dictionary without changing the rest of the application.

1.2.2 Structural Design Patterns

Structural design patterns are about how classes and objects are composed to form larger structures. They focus on simplifying the structure of a system by identifying a simple way to realize relationships between entities. These patterns help to reduce complexity by organizing objects and classes into larger, more cohesive structures. They often use concepts like inheritance and composition to build new functionalities (GeeksforGeeks, 2023).

a) Proxy method Design pattern

The Proxy design pattern provides a surrogate or placeholder for another object to control access to it. It acts as an intermediary, forwarding requests to the real object. This pattern is useful for adding a layer of control, such as access management, lazy initialization, or logging, without altering the real object's code (GeeksforGeeks, 2017).

```
# Example of Proxy usage: Streamlit mediates between your app and the UI
figures = analyzer.visualize(analysis_result)
st.plotly_chart(figures['hourly'], use_container_width=True)

# Another proxy example for showing metrics
st.metric(
    label="📊 Total Passengers",
    value=f"{len(data):,}",
    delta=f"{len(data)} records"
)
```

Figure 12: Coding example for Adapter method Design pattern

Code Explanation: The application uses **Streamlit functions** like `st.plotly_chart`, `st.metric`, and `st.dataframe`. These act as **Proxies** because they sit between your analyzer logic and the actual rendering engine. This code doesn't directly manage rendering; it delegates it through Streamlit, which controls how the chart or data is shown to users.

Impact

- Decouples application logic from the GUI rendering engine.
- Allows analyzers to remain independent of the front-end framework.

- If the framework changes (e.g., switch from Streamlit to Flask+Dash), analyzers remain untouched.

b) Facade method design pattern

The Facade pattern provides a simplified interface to a complex system of classes, libraries, or frameworks. It hides the complexity of the subsystem from the client by providing a single, unified entry point. This makes the system easier to use and reduces dependencies (GeeksforGeeks, 2017).

```
class BusPassengerAnalysisApp:
    def run(self):
        self.setup_page_config()

        # Load and preprocess data
        data = self.load_data("bus_passenger_dataset.csv")

        # Apply filters
        filters = self.create_sidebar_filters(data)
        filtered_data = self.apply_filters(data, filters)

        # Display dashboards and results
        self.display_overview_metrics(filtered_data)
        analyzer = self.analyzers['congestion']
        analysis_result = analyzer.analyze(filtered_data)
        figures = analyzer.visualize(analysis_result)
        st.plotly_chart(figures['hourly'], use_container_width=True)
```

Figure 13: Coding example for Façade method design pattern

Code Explanation: The BusPassengerAnalysisApp class acts as a Facade because it provides a single-entry point (run()) that hides the complexity of multiple subsystems

- Data loading (DataProcessor)
- Filtering (SearchEngine + filters)
- Analysis (CongestionAnalyzer, RouteAnalyzer, etc.)
- Visualization (Plotly + Streamlit)

Users don't need to know the details of how these work internally—they just interact with the app.

Impact

- Simplifies the user's interaction with the system.
- Reduces coupling between the UI and internal subsystems.
- Makes the system easier to maintain and extend, since changes inside analyzers or filters don't affect the interface.

1.2.3 Behavioural Design Patterns

Behavioural design patterns are concerned with how objects interact and distribute responsibility. They describe communication patterns between objects, making the system more flexible and reusable. These patterns focus on the algorithms and the assignment of responsibilities between objects. They help in designing objects that are not only responsible for their own state but also for their interactions with other objects.

a) Strategy method design pattern

The Strategy pattern defines a family of algorithms, encapsulates each one, and makes them interchangeable. It lets the algorithm vary independently from the clients that use it. This allows for a flexible design where a client can choose a behaviour at runtime (Shah, 2024).

```
class SearchEngine:
    def __init__(self):
        self._strategies = {
            'time': TimeFilter(),
            'region': RegionFilter(),
            'route': RouteFilter()
        }

    def search(self, data: pd.DataFrame, search_type: str, **kwargs) -> pd.DataFrame:
        if search_type in self._strategies:
            return self._strategies[search_type].filter(data, **kwargs)
        return data
```

Figure 14: Coding example for Strategy method design pattern

Code Explanation: The SearchEngine implements the **Strategy pattern**. Different filtering algorithms are encapsulated as separate classes (TimeFilter, RegionFilter, RouteFilter), and SearchEngine chooses which one to apply based on user input.

Impact

- Makes filtering logic interchangeable without modifying the main app.
- New filtering strategies (e.g., FareFilter, StationFilter) can be added easily.
- Improves flexibility and adheres to the **Open/Closed Principle** (extend without modifying existing code).

b) Template method design pattern

The Template Method pattern defines the skeleton of an algorithm in a base class but lets subclasses override specific steps of the algorithm without changing its overall structure. It's a way to reuse code and ensure that the structure of an operation remains consistent across different implementations (GeeksforGeeks, 2017).

```
class DataAnalyzer(ABC):
    @abstractmethod
    def analyze(self, data: pd.DataFrame) -> Dict:
        pass

    @abstractmethod
    def visualize(self, analysis_result: Dict) -> go.Figure:
        pass

class CongestionAnalyzer(DataAnalyzer):
    def analyze(self, data: pd.DataFrame) -> Dict:
        hourly_congestion = data.groupby('hour').size().reset_index(name='passenger_count')
        return {'hourly': hourly_congestion}

    def visualize(self, analysis_result: Dict) -> Dict[str, go.Figure]:
        fig = px.bar(analysis_result['hourly'], x='hour', y='passenger_count')
        return {'hourly': fig}
```

Figure 15: Coding example for Template method design pattern

Code Explanation: The DataAnalyzer abstract base class demonstrates the **Template Method pattern**. It defines the structure for analyzers with the methods analyze and visualize. Subclasses (CongestionAnalyzer, RouteAnalyzer, DisruptionAnalyzer, RegionalAnalyzer) provide their own implementations for these steps, but the overall workflow remains consistent.

Impact

- Ensures all analyzers follow the same blueprint.
- Makes the system extensible: new analyzers can be created by implementing the same template.
- Promotes code reusability and consistency across multiple modules.

1.3 The impact of clean coding techniques on the use of data structures and operations when writing algorithms

Clean coding is a set of principles and practices that make software code readable, maintainable, and efficient. It emphasizes writing code in a way that is easy to understand for both the original author and future developers who may maintain the system. In large-scale applications such as the Bus Passenger Data Analysis Application, clean coding becomes critical since the software handles big data processing, analysis, and visualization.

One of the most important areas impacted by clean coding techniques is the use of data structures and algorithms. Choosing the right data structures and applying clean coding practices directly influences the performance, scalability, and maintainability of algorithms.

1. Meaningful Naming

Using descriptive and meaningful names for variables, methods, and classes improves readability and ensures clarity in data structure usage (Codacy, 2023).

- **Impact on Data Structures:** Instead of generic names like `arr` or `x`, meaningful names like `passenger_data`, `filtered_routes`, or `hourly_congestion` make it clear what data is being stored and processed.
- **Impact on Algorithms:** Helps developers understand algorithm logic quickly without deep diving into the implementation. For example, an aggregation algorithm over `route_popularity` is more intuitive than over a vague `df1`.

Before (bad code – unclear names):

```
x = pd.read_csv("data.csv")
y = x.groupby("r")["p"].count()
```

Figure 16: Before using Meaningful Naming technique

After (clean code – meaningful names):

```
passenger_data = pd.read_csv("bus_passenger_data.csv")
route_popularity = passenger_data.groupby("route_id")["passenger_id"].count()
```

Figure 17: after using Meaningful naming technique

2. Single Responsibility Principle (SRP)

Each class or method should focus on a single responsibility (zen8labs, 2024).

- **Impact on Data Structures:** Data operations are clearly separated. For instance, DataProcessor handles preprocessing while analyzers handle analysis, ensuring no overlap.
- **Impact on Algorithms:** Algorithms remain focused and optimized. A method responsible for filtering will not mix in visualization logic, making data traversal more efficient and debugging easier.

Before (bad code – mixing responsibilities):

```
def process_data(file):  
    df = pd.read_csv(file)  
    df['timestamp'] = pd.to_datetime(df['timestamp'])  
    grouped = df.groupby("route")["id"].count()  
    plt.plot(grouped)    # visualization mixed with processing  
    return grouped
```

Figure 18: Before using Single Responsibility Principle (SRP)

After (clean code – separate responsibilities):

```
def load_and_preprocess(file):  
    df = pd.read_csv(file)  
    df['timestamp'] = pd.to_datetime(df['timestamp'])  
    return df  
  
def analyze_routes(data):  
    return data.groupby("route_id")["passenger_id"].count()  
  
def visualize_routes(route_stats):  
    plt.plot(route_stats)
```

Figure 19: after using Single Responsibility Principle (SRP)

3. Consistent Formatting and Code Structure

Consistent indentation, spacing, and structure (like proper method grouping) makes algorithms easier to follow (Codacy, 2023).

- **Impact on Data Structures:** Developers can quickly spot how lists, dictionaries, or DataFrames are created, modified, or returned.
- **Impact on Algorithms:** Complex operations such as filtering or aggregating large datasets become easier to trace, minimizing logical errors and improving debugging.

Before (bad code – messy formatting):

```
df=pd.read_csv("data.csv");df['time']=pd.to_datetime(df['time']);routeStats=df.groupby("route")["
```

Figure 20: before using Consistent Formatting and Code Structure

After (clean code – consistent style):

```
df = pd.read_csv("bus_data.csv")
df['timestamp'] = pd.to_datetime(df['timestamp'])

route_stats = df.groupby("route_id")["passenger_id"].count()
```

Figure 21: after using Consistent Formatting and Code Structure

4. Modularity and Reusability

Breaking large tasks into smaller reusable functions and classes improves maintainability.

- **Impact on Data Structures:** Data structures like DataFrame are passed between functions instead of being duplicated, reducing memory usage in large datasets.
- **Impact on Algorithms:** Algorithms like grouping, filtering, or aggregating are implemented once and reused across analyzers, reducing redundancy and improving reliability.

Before (bad code – duplicated logic):

```
df1 = pd.read_csv("file1.csv")
df1['hour'] = pd.to_datetime(df1['timestamp']).dt.hour

df2 = pd.read_csv("file2.csv")
df2['hour'] = pd.to_datetime(df2['timestamp']).dt.hour
```

Figure 22: before using Modularity and Reusability

After (clean code – reusable function):

```
def load_with_hour(file):
    df = pd.read_csv(file)
    df['hour'] = pd.to_datetime(df['timestamp']).dt.hour
    return df

df1 = load_with_hour("file1.csv")
df2 = load_with_hour("file2.csv")
```

Figure 23: after using Modularity and Reusability

5. Error Handling and Robustness

Clean code handles exceptions gracefully without breaking execution (zen8labs, 2024).

- **Impact on Data Structures:** Ensures corrupt or incomplete data structures (like missing values in a dataset) are handled safely. For instance, filling missing values in transfer stations avoids algorithm failures.
- **Impact on Algorithms:** Prevents runtime errors in iterative or recursive algorithms, especially when processing big datasets in chunks. This ensures system stability and reliable performance.

Before (bad code – no error handling):

```
data = pd.read_csv("missing_file.csv")
data['timestamp'] = pd.to_datetime(data['timestamp'])
```

Figure 24: before using Error Handling and Robustness

After (clean code – robust error handling):

```
try:
    data = pd.read_csv("bus_passenger_data.csv")
    data['timestamp'] = pd.to_datetime(data['timestamp'], errors='coerce')
except FileNotFoundError:
    print("Error: Dataset not found. Please check the file path.")
```

Figure 25: after using Error Handling and Robustness

6. Avoiding Code Duplication

Algorithms and data operations should not be repeated unnecessarily (Codacy, 2023).

- **Impact on Data Structures:** Shared operations like converting timestamps or extracting features (hour, day, month) are centralized in DataProcessor, so they aren't redundantly applied in every analyzer.
- **Impact on Algorithms:** Reduces algorithmic complexity and improves performance by avoiding repeated operations on large datasets.

Before (bad code – repeated feature extraction):

```
df = pd.read_csv("bus.csv")
df['hour'] = pd.to_datetime(df['timestamp']).dt.hour
df['day'] = pd.to_datetime(df['timestamp']).dt.day

df['month'] = pd.to_datetime(df['timestamp']).dt.month
df['year'] = pd.to_datetime(df['timestamp']).dt.year
```

Figure 26: before using Avoiding Code Duplication

After (clean code – centralized in one function):

```
def extract_time_features(data):  
    data['timestamp'] = pd.to_datetime(data['timestamp'])  
    data['hour'] = data['timestamp'].dt.hour  
    data['day'] = data['timestamp'].dt.day  
    data['month'] = data['timestamp'].dt.month  
    data['year'] = data['timestamp'].dt.year  
    return data  
  
df = pd.read_csv("bus.csv")  
df = extract_time_features(df)
```

Figure 27: after using Avoiding Code Duplication

7. Readability and Documentation

Adding comments and docstrings improves code comprehension for future developers (zen8labs, 2024).

- **Impact on Data Structures:** Describes why certain data structures (like dictionaries for analyzers, DataFrames for passenger data) are used, justifying their efficiency.
- **Impact on Algorithms:** Provides clarity in algorithm steps (e.g., how anomalies are detected or congestion is calculated), making it easier to optimize or extend in the future.

Before (bad code – no comments, hard to follow):

```
df = pd.read_csv("bus.csv")  
df['t'] = pd.to_datetime(df['t'])  
res = df.groupby("r")["p"].count()
```

Figure 28: before using Readability and Documentation

After (clean code – with comments and docstrings):

```
def calculate_route_popularity(data: pd.DataFrame) -> pd.DataFrame:
    """
    Calculates the number of passengers per route.

    Args:
        data (pd.DataFrame): Passenger dataset with route_id and passenger_id.

    Returns:
        pd.DataFrame: A DataFrame with passenger counts grouped by route_id.
    """
    data['timestamp'] = pd.to_datetime(data['timestamp'])
    route_popularity = data.groupby("route_id")["passenger_id"].count()
    return route_popularity

df = pd.read_csv("bus.csv")
route_stats = calculate_route_popularity(df)
```

Figure 29: after using Readability and Documentation

Clean coding techniques are not just about style; they directly affect how effectively data structures and algorithms are implemented. In the Bus Passenger Data Analysis Application, these techniques ensure that the system can handle large-scale passenger datasets efficiently while remaining modular, maintainable, and extensible. As the dataset grows or new analytical requirements arise, the impact of clean coding becomes even more significant for long-term success.

1.4 Evaluation on the Impact of SOLID Principles in Object-Oriented Application Development

The SOLID principles - an acronym coined by Robert C. Martin - represent five foundational tenets of object-oriented design aimed at improving maintainability, extensibility, testability, and scalability of software systems. Here's a structured breakdown of each principle:

1.4.1. Single Responsibility Principle (SRP)

Each class or module should have **one, and only one, reason to change**, meaning it should focus on a single responsibility. Violating SRP often leads to monolithic “God classes” that bundle unrelated functionality (Ramos, 2023).

Impact:

- Enhances **maintainability**—small, focused classes are easier to understand and modify.
- Improves **testability**—isolating behavior makes unit testing simpler.
- Reduces **coupling**, since changing one aspect doesn't ripple through the rest of the system.

Coding Example:

```
# Violating SRP:
class FileManager:
    def read(self): pass
    def write(self): pass
    def compress(self): pass
    def decompress(self): pass

# SRP-compliant design:
class FileReaderWriter:
    def read(self): ...
    def write(self): ...

class ZipFileManager:
    def compress(self): ...
    def decompress(self): ...
```

Figure 30: Coding example for Single Responsibility Principle (SRP)

Here, responsibilities are cleanly separated into smaller, more focused classes

1.4.2. Open–Closed Principle (OCP)

Software entities (classes, modules, functions, etc.) should be open for extension but closed for modification, you should add behaviour without altering existing code (Ramos, 2023).

Impact:

- Promotes **extensibility** and reduces the risk of regressions.
- Encourages **safe scaling**—new features can be added with minimal disruption to stable code.

Coding Example:

```
from abc import ABC, abstractmethod

class Shape(ABC):
    @abstractmethod
    def area(self):
        pass

class Circle(Shape):
    def area(self): return ...

class Rectangle(Shape):
    def area(self): return ...
```

Figure 31: Coding example for Open-Closed Principle (OCP)

To add a new shape, you extend the base Shape class without modifying existing implementations—a clear application of the OCP

1.4.3. Liskov Substitution Principle (LSP)

Subtypes must be substitutable for their base types without altering the correctness of the program. Essentially, the behaviour of derived classes should align with expectations established by the parent class (Özateş, 2022).

Impact:

- Enhances **polymorphism** and ensures **reliable substitution**.
- Preserves **program correctness**—no unexpected behavior when replacing parent with child.

Coding Example:

```
class Rectangle:
    def set_width(self, w): ...
    def set_height(self, h): ...

class Square(Rectangle):
    def set_width(self, w):
        self.width = self.height = w # Violates LSP
```

Figure 32: Coding example for Liskov Substitution Principle (LSP)

A Square breaks LSP by changing behaviour of `set_width`; replacing a `Rectangle` reference with a `Square` leads to unexpected behaviour.

1.4.4. Interface Segregation Principle (ISP)

Clients should not be forced to depend on methods they do not use. Favor smaller, specific interfaces rather than monolithic ones (umaparvathy, 2023).

Impact:

- Promotes **decoupling**—clients only rely on what's necessary.
- Encourages **flexibility** and **clarity**, reducing overhead for implementers.

Coding Example:

```
# Violation:
class IShape:
    def draw(self): ...
    def resize(self): ...
    def rotate(self): ...

# ISP-compliant segregation:
class Drawable:
    def draw(self): ...

class Resizable:
    def resize(self): ...

class Rotatable:
    def rotate(self): ...
```

Figure 33: Coding example for Interface Segregation Principle (ISP)

Clients only implement needed capabilities, keeping dependencies minimal.

1.4.5. Dependency Inversion Principle (DIP)

High-level modules should not depend on low-level modules but on **abstractions**. Additionally, abstractions should not depend on details—details should depend on abstractions (umaparvathy, 2023).

Impact:

- Decreases **coupling**, making components easier to replace or extend.
- Enhances **testability** through dependency injection of mock implementations.
- Improves **flexibility**—you can easily swap implementations without touching high-level logic.

Coding Example:

```
from abc import ABC, abstractmethod

class DataSource(ABC):
    @abstractmethod
    def fetch(self): pass

class Database(DataSource):
    def fetch(self): ...

class API(DataSource):
    def fetch(self): ...

class FrontEnd:
    def __init__(self, source: DataSource):
        self.source = source
    def display(self):
        data = self.source.fetch()
        # render data
```

Figure 34: Coding example for Dependency Inversion Principle (DIP)

Here, FrontEnd depends on the abstraction DataSource, not concrete implementations—facilitating extension and testing.

Part 2

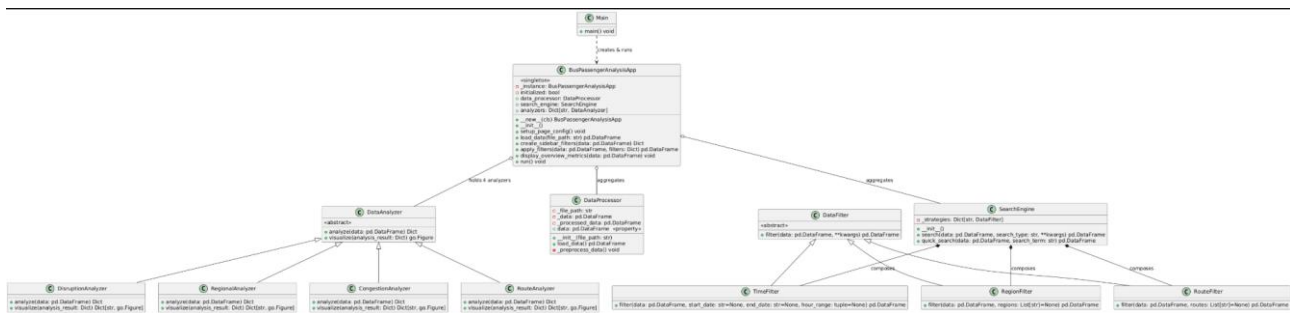


Figure 35: UML class diagram before refinements

The initial UML class diagram provides a structural view of the system without emphasizing specific design patterns. It identifies the core components such as **DataProcessor** (responsible for loading and preprocessing data), **SearchEngine** (responsible for filtering and searching), and a set of specialized **DataAnalyzer** subclasses (for congestion, route, disruption, and regional analysis). The diagram also highlights the main relationships among components: aggregation of analyzers within the main application class, and the use of multiple filter classes by the search engine. This version serves as a straightforward documentation of the system's classes, attributes, and interactions but does not explicitly capture higher-level design principles.

2.1 Utilization of SOLID Principles, Clean Coding Techniques, and Design Patterns in the Proposed Application

2.1.1 Utilization of SOLID Principles

The proposed bus passenger analysis system has been designed with adherence to the **SOLID principles**, ensuring scalability, flexibility, and maintainability.

- **Single Responsibility Principle (SRP):**

Each class in the system has a well-defined responsibility. For example, the **DataProcessor** class is solely responsible for data loading and preprocessing, while **SearchEngine** handles filtering and search logic. Similarly, each **DataAnalyzer** subclass (e.g., **CongestionAnalyzer**, **RouteAnalyzer**) is responsible for analyzing a specific aspect of the data.

- **Open/Closed Principle (OCP):**

The design allows extension without modifying existing classes. For instance, new analyzer types can be introduced by simply creating new subclasses of **DataAnalyzer**. Likewise, additional filters can be added by extending the **DataFilter** abstraction without altering the **SearchEngine** logic.

- **Liskov Substitution Principle (LSP):**

All subclasses of **DataAnalyzer** and **DataFilter** can be substituted wherever their parent class is expected. This ensures consistent behavior and interchangeable use of analyzers and filters.

- **Interface Segregation Principle (ISP):**

Abstract classes such as **DataAnalyzer** and **DataFilter** expose only the methods necessary for their specific functionality. This prevents subclasses from being forced to implement methods that are irrelevant to them.

- **Dependency Inversion Principle (DIP):**

High-level modules such as **BusPassengerAnalysisApp** depend on abstractions (**DataAnalyzer**, **DataFilter**) rather than concrete implementations. This makes the system flexible and reduces tight coupling between components.

2.1.2 Utilization of Clean Coding Techniques

The application also applies **clean coding techniques** to ensure readability, maintainability, and ease of collaboration.

- **Meaningful Naming Conventions:**

Class names such as **DataProcessor**, **SearchEngine**, and **RouteAnalyzer** clearly describe their responsibilities. Variable and method names are self-explanatory, minimizing ambiguity.

- **Code Modularization:**

The system is divided into smaller, reusable modules. Each class and method serves a single, focused purpose, reducing complexity and increasing readability.

- **Encapsulation and Information Hiding:**

Private attributes (e.g., **_data**, **_file_path**) restrict direct access, and public methods provide controlled interaction with class internals. This protects data integrity and enforces clear boundaries.

- **Consistency and Style:**

The code follows a consistent style in terms of formatting, naming, and method structuring. This ensures uniformity across the system and makes the application easier to extend or debug.

- **Error Handling and Robustness:**

Methods like **load_data()** and filtering functions are designed to handle invalid inputs gracefully, increasing system reliability.

3.1.3 Utilization of Design Patterns

The proposed system incorporates several **design patterns** to improve flexibility and maintainability.

- **Singleton Pattern:**

The **BusPassengerAnalysisApp** class is implemented as a <<**Singleton**>>, ensuring that only one instance of the main application exists throughout execution. This centralizes system control and resource management.

- **Strategy Pattern:**

The **SearchEngine** class uses the <<**Strategy**>> pattern by delegating search operations to interchangeable filter strategies such as **TimeFilter**, **RegionFilter**, and **RouteFilter**. This provides flexibility to add or replace filtering strategies without modifying core logic.

- **Template Method Pattern:**

The **DataAnalyzer** abstract class follows the <<**Template Method**>> pattern, defining the structure of analysis and visualization while allowing subclasses (e.g., **CongestionAnalyzer**, **RouteAnalyzer**) to implement specific behaviors. This promotes consistency across analyzers while supporting specialization.

- **Facade Pattern:**

The **BusPassengerAnalysisApp** also acts as a <<**Facade**>> by providing a simplified interface (**run()**, **apply_filters()**, **display_overview_metrics()**) to interact with complex subsystems such as the **DataProcessor**, **SearchEngine**, and various analyzers. This hides implementation details and makes the application easier to use and maintain.

- **Factory Method Pattern:**

Analyzer instantiation within **BusPassengerAnalysisApp** reflects the use of the <<**Factory Method**>> pattern. By maintaining analyzers in a dictionary and initializing them dynamically, the system supports seamless addition of new analyzers without modifying client code.

- **Extensibility for Future Patterns:**

The modular design also supports the introduction of additional patterns in future development. For example, a **Factory Method** could be introduced for dynamic analyzer creation, or an **Observer** pattern could be applied to update the UI automatically when new data is analyzed.

By carefully applying **SOLID principles, clean coding techniques, and established design patterns**, the proposed application achieves high modularity, maintainability, and extensibility. The system design not only addresses current functional requirements but also establishes a strong foundation for future enhancements and scalability.

2.2 Explanation about how the design of the proposed application is refined to include multiple design patterns.

The initial design of the proposed bus passenger analysis application was centered on the basic class structure, highlighting the main components such as the **DataProcessor**, **SearchEngine**, and various **DataAnalyzer** subclasses. While this unrefined design documented the relationships and responsibilities of classes, it did not explicitly capture the underlying design principles that could improve flexibility, scalability, and maintainability.

To refine the system design, **multiple design patterns** were incorporated and clearly mapped to different parts of the application. This refinement ensures that the architecture not only represents structural relationships but also demonstrates intentional use of reusable and well-established solutions to common software problems.

- **Singleton Pattern:**

The **BusPassengerAnalysisApp** was refined into a **<<Singleton>>**, ensuring centralized control by restricting instantiation to a single object. This refinement prevents conflicts that could arise from multiple application instances.

- **Strategy Pattern:**

The search functionality was enhanced using the <<**Strategy**>> pattern, where the **SearchEngine** can dynamically choose between different filtering strategies (**TimeFilter**, **RegionFilter**, **RouteFilter**). This refinement decouples the filtering logic from the search engine and makes the system easier to extend with new search strategies.

- **Template Method Pattern:**

The design of analyzers was refined using the <<**Template Method**>> pattern. The **DataAnalyzer** abstract class now defines the standard framework for analysis and visualization, while its subclasses override specific steps. This refinement enforces consistency while allowing specialized behaviors.

- **Factory Method Pattern:**

The instantiation of analyzers within the main application was further refined to use a <<**Factory Method**>> approach. By maintaining analyzers in a dictionary and constructing them dynamically, the system supports easy extension with new analyzers without modifying client code.

- **Facade Pattern:**

Finally, the role of the `BusPassengerAnalysisApp` itself was refined into a <<**Facade**>>, exposing simple methods such as **run()**, **apply_filters()**, and **display_overview_metrics()** while hiding the complexity of the underlying subsystems. This refinement makes the application easier to use and enhances maintainability.

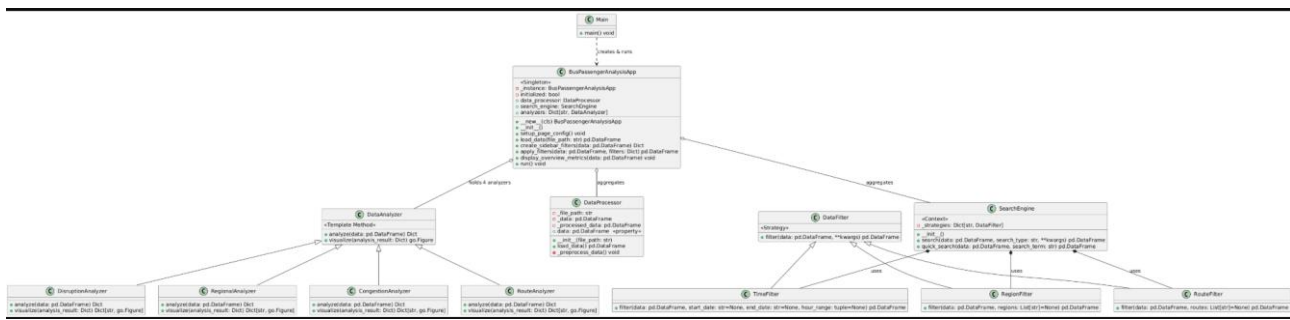


Figure 36: UML class diagram after the refinements

The refined UML class diagram improves upon the initial structure by explicitly highlighting the **design patterns** that shape the system. The **BusPassengerAnalysisApp** class is shown as a **<<Singleton>>**, ensuring that only one instance of the application exists throughout execution. The **SearchEngine** class implements the **<<Strategy>>** pattern, delegating filtering logic to interchangeable filter classes such as **TimeFilter**, **RegionFilter**, and **RouteFilter**.

The **DataAnalyzer** hierarchy adopts the **<<Template Method>>** pattern, where the abstract class defines a framework for analysis and visualization, while concrete subclasses implement specific behaviours. By pointing out these patterns, the refined diagram emphasizes not just the structure but also the intentional design choices that make the system more flexible, extensible, and maintainable.

Through these refinements, the application design evolved from a basic representation of classes into a **pattern-oriented architecture** that is robust, modular, and future-proof. By combining multiple design patterns, the proposed system is better positioned to handle complexity, support extension, and provide a clean separation of concerns.

2.3 Designing a test plan (testing regime) for the proposed application by including a provision for automated testing.

2.3.1 Testing Strategies

The proposed application adopts a **hybrid testing strategy** that combines **manual testing** and **automated testing** to ensure correctness, reliability, and maintainability.

- **Unit Testing:** Each component (e.g., DataProcessor, SearchEngine, DataAnalyzer subclasses) is tested independently using automated frameworks such as **pytest** to verify correctness of individual methods.
- **Integration Testing:** Ensures that modules interact correctly, e.g., verifying that the BusPassengerAnalysisApp correctly integrates analyzers, filters, and data processor.
- **Functional Testing:** Validates whether the system features (e.g., congestion analysis, disruption detection) work as intended against user requirements.
- **Regression Testing:** Automated tests are re-executed whenever code changes to ensure new updates do not break existing features.
- **Performance Testing (Optional):** Evaluates application responsiveness when processing large datasets.

2.3.2 Test Plan for the Proposed Application

- **Content**
 - ✓ Objectives
 - ✓ Objects to be tested
 - ✓ System features
 - ✓ Schedule of the testing
 - ✓ Tools used for the testing

1. Objectives

- To verify that all components of the bus passenger analysis application work as expected.
- To ensure that the system features (congestion analysis, route analysis, disruption detection, regional analysis) produce accurate and reliable results.

- To validate integration of different modules (data loading, filtering, searching, analyzing, and visualization).
- To confirm maintainability by introducing automated testing using **pytest**, allowing repeatable and efficient test execution.

2. Objects to be Tested

- Peak Congestion Analysis – correctness of passenger congestion detection.
- Popular Route and Transfer Point Analysis – accuracy in identifying most used routes and transfer points.
- Service Disruption Detection – ability to flag abnormal or missing services.
- Regional Performance Analysis – evaluation of service quality and reliability across regions.

3. System Features

Hardware Requirements:

- **RAM:** 8 GB minimum (16 GB recommended).
- **Hard Disk:** 500 MB for application and dataset storage.
- **Processor:** Intel i5 / AMD Ryzen 5 (or higher).

Software Requirements:

- **Operating System:** Windows 10 / Linux Ubuntu 20.04 / macOS Ventura.
- **Python IDE:** PyCharm / VS Code / Jupyter Notebook.
- **Python Packages and Libraries:**
 - **pandas, numpy** (data handling & preprocessing)
 - **plotly, matplotlib** (visualization)
 - **pytest, unittest** (testing framework)
 - **streamlit** (application interface)

4. Schedule of the Testing (Unit Tests using pytest)

Task	Start Date	End Date
Write unit tests for DataProcessor	01-Aug-2025	03-Aug-2025
Write unit tests for DataFilter & SearchEngine	04-Aug-2025	07-Aug-2025
Write unit tests for DataAnalyzer subclasses	08-Aug-2025	12-Sep-2025
Integration testing of modules	13-Aug-2025	15-Aug-2025
Functional testing of full application	16-Aug-2025	18-Aug-2025
Regression testing (automated re-runs)	19-Aug-2025	20-Aug-2025

Table 1: Testing Schedule

5. Tools Used for the Testing

- **Python Libraries:**
 - ✓ **pytest** → Unit and integration testing
 - ✓ **pandas & numpy** → Test dataset creation and validation
 - ✓ **coverage.py** → Code coverage reporting

Part 3

3.1 Source Code of the Developed System with evidence.

In this part I attached my source code screenshots as evidence of the system development as proposed. The complete source code attached along with the report.

3.1.1 Core Abstractions

a) DataAnalyzer (analysis contract)

Defines the analysis interface used across all analyzers. This abstraction enables consistent pipelines and easy extension with new analyzers.

```
35 class DataAnalyzer(ABC):
36     """Abstract base class for all data analyzers - demonstrates abstraction"""
37
38     @abstractmethod
39     def analyze(self, data: pd.DataFrame) -> Dict:
40         """Abstract method that must be implemented by all analyzers"""
41         pass
42
43     @abstractmethod
44     def visualize(self, analysis_result: Dict) -> go.Figure:
45         """Abstract method for creating visualizations"""
46         pass
```

Figure 37: DataAnalyzer class

b) DataFilter (filtering contract) + concrete strategies

A strategy interface (DataFilter) plus three concrete strategies (time/region/route). The SearchEngine can swap these without changing its own code.

```
48 class DataFilter(ABC):
49     """Abstract base class for data filtering strategies"""
50
51     @abstractmethod
52     def filter(self, data: pd.DataFrame, **kwargs) -> pd.DataFrame:
53         """Abstract method for filtering data"""
54         pass
```

Figure 38: DataFilter class

```

60 class TimeFilter(DataFilter):
61     """Concrete filter for time-based filtering"""
62
63     def filter(self, data: pd.DataFrame, start_date: str = None, end_date: str = None,
64               hour_range: tuple = None) -> pd.DataFrame:
65         filtered_data = data.copy()
66
67         if start_date:
68             filtered_data = filtered_data[filtered_data['timestamp'] >= start_date]
69         if end_date:
70             filtered_data = filtered_data[filtered_data['timestamp'] <= end_date]
71         if hour_range:
72             filtered_data = filtered_data[
73                 (filtered_data['hour'] >= hour_range[0]) &
74                 (filtered_data['hour'] <= hour_range[1])
75             ]
76
77         return filtered_data

```

Figure 39: TimeFilter Class

```

79 class RegionFilter(DataFilter):
80     """Concrete filter for region-based filtering"""
81
82     def filter(self, data: pd.DataFrame, regions: List[str] = None) -> pd.DataFrame:
83         if regions:
84             return data[data['region'].isin(regions)]
85         return data
86

```

Figure 40: RegionFilter class

```

87 class RouteFilter(DataFilter):
88     """Concrete filter for route-based filtering"""
89
90     def filter(self, data: pd.DataFrame, routes: List[str] = None) -> pd.DataFrame:
91         if routes:
92             return data[data['route_id'].isin(routes)]
93         return data

```

Figure 41: RouteFilter class

3.1.2 Data Access & Preprocessing

a) DataProcessor (repository for datasets)

Encapsulates file I/O and preprocessing (parsing dates/hours). Exposes a clean data property used by analyzers and search.

```
99 class DataProcessor:
100     """Handles all data loading and preprocessing operations"""
101
102     def __init__(self, file_path: str):
103         self._file_path = file_path
104         self._data = None
105         self._processed_data = None
106
107     def load_data(self) -> pd.DataFrame:
108         """Load data from CSV file with error handling"""
109         try:
110             self._data = pd.read_csv(self._file_path)
111             self._preprocess_data()
112             return self._processed_data
113         except Exception as e:
114             st.error(f"Error loading data: {str(e)}")
115             return pd.DataFrame()
116
117     def _preprocess_data(self):
118         """Private method to preprocess the loaded data"""
119         if self._data is not None:
120             # Convert timestamp columns
121             self._data['timestamp'] = pd.to_datetime(self._data['timestamp'])
122             self._data['boarding_time'] = pd.to_datetime(self._data['boarding_time'])
123             self._data['alighting_time'] = pd.to_datetime(self._data['alighting_time'])
124
125             # Extract time components for analysis
126             self._data['hour'] = self._data['timestamp'].dt.hour
127             self._data['day'] = self._data['timestamp'].dt.day
128             self._data['month'] = self._data['timestamp'].dt.month
129             self._data['year'] = self._data['timestamp'].dt.year
130             self._data['date'] = self._data['timestamp'].dt.date
131
132             # Handle missing values in transfer_station_id
133             self._data['transfer_station_id'] = self._data['transfer_station_id'].fillna('No Transfer')
134
135             self._processed_data = self._data
136
137     @property
138     def data(self) -> pd.DataFrame:
139         """Getter for processed data - demonstrates encapsulation"""
140         return self._processed_data if self._processed_data is not None else pd.DataFrame()
```

Figure 42: DataProcessor class

3.1.3 Search & Filtering Orchestration

a) SearchEngine (context that applies strategies)

Holds a registry of filter strategies and applies them via `search()`. `quick_search()` provides fuzzy term-based filtering.

```
391 class SearchEngine:
392     """Implements various search algorithms for data filtering"""
393
394     def __init__(self):
395         self._strategies = {
396             'time': TimeFilter(),
397             'region': RegionFilter(),
398             'route': RouteFilter()
399         }
400
401     def search(self, data: pd.DataFrame, search_type: str, **kwargs) -> pd.DataFrame:
402         """Apply search strategy based on type"""
403         if search_type in self._strategies:
404             return self._strategies[search_type].filter(data, **kwargs)
405         return data
406
407     def quick_search(self, data: pd.DataFrame, search_term: str) -> pd.DataFrame:
408         """Quick text search across multiple columns"""
409         if not search_term:
410             return data
411
412         # Convert search term to string and make case-insensitive
413         search_term = str(search_term).lower()
414
415         # Search across relevant string columns
416         mask = (
417             data['passenger_id'].astype(str).str.lower().str.contains(search_term, na=False) |
418             data['route_id'].astype(str).str.lower().str.contains(search_term, na=False) |
419             data['region'].astype(str).str.lower().str.contains(search_term, na=False) |
420             data['service_status'].astype(str).str.lower().str.contains(search_term, na=False)
421         )
422
423         return data[mask]
```

Figure 43: SearchEngine class

3.1.4 Analytical Modules

a) CongestionAnalyzer

Aggregates passenger counts by hour and identifies top peak hours. Returns Plotly figures for dashboard use.

```
146 class CongestionAnalyzer(DataAnalyzer):
147     """Analyzes peak congestion patterns - implements DataAnalyzer interface"""
148
149     def analyze(self, data: pd.DataFrame) -> Dict:
150         """Analyze congestion patterns by different time periods"""
151         analysis = {}
152
153         # Hourly congestion
154         hourly_congestion = data.groupby('hour').size().reset_index(name='passenger_count')
155         analysis['hourly'] = hourly_congestion
156
157         # Daily congestion
158         daily_congestion = data.groupby('day_of_week').size().reset_index(name='passenger_count')
159         analysis['daily'] = daily_congestion
160
161         # Monthly congestion
162         monthly_congestion = data.groupby('month').size().reset_index(name='passenger_count')
163         analysis['monthly'] = monthly_congestion
164
165         # Station congestion (entry and exit)
166         entry_congestion = data.groupby('entry_station_id').size().reset_index(name='entries')
167         exit_congestion = data.groupby('exit_station_id').size().reset_index(name='exits')
168         analysis['stations'] = {'entries': entry_congestion, 'exits': exit_congestion}
169
170         return analysis
```

Figure 44: CongestionAnalyzer class 1

```

172 def visualize(self, analysis_result: Dict) -> Dict[str, go.Figure]:
173     """Create visualizations for congestion analysis"""
174     figures = {}
175
176     # Hourly congestion chart
177     fig_hourly = px.bar(
178         analysis_result['hourly'],
179         x='hour',
180         y='passenger_count',
181         title='🕒 Peak Hours Analysis - Passenger Count by Hour',
182         color='passenger_count',
183         color_continuous_scale='Viridis'
184     )
185     fig_hourly.update_layout(
186         xaxis_title="Hour of Day",
187         yaxis_title="Number of Passengers",
188         template="plotly_white"
189     )
190     figures['hourly'] = fig_hourly
191
192     # Daily congestion chart
193     fig_daily = px.pie(
194         analysis_result['daily'],
195         values='passenger_count',
196         names='day_of_week',
197         title='📅 Weekly Pattern - Passenger Distribution by Day'
198     )
199     figures['daily'] = fig_daily
200
201     # Monthly congestion chart
202     fig_monthly = px.line(
203         analysis_result['monthly'],
204         x='month',
205         y='passenger_count',
206         title='📊 Monthly Trends - Passenger Count by Month',
207         markers=True
208     )
209     fig_monthly.update_layout(template="plotly_white")
210     figures['monthly'] = fig_monthly
211
212     return figures

```

Figure 45: CongestionAnalyzer class 2

b) RouteAnalyzer

Finds popular routes and transfer hubs; provides visuals for each.

```
214 class RouteAnalyzer(DataAnalyzer):
215     """Analyzes popular routes and transfer points"""
216
217     def analyze(self, data: pd.DataFrame) -> Dict:
218         """Analyze route popularity and transfer patterns"""
219         analysis = {}
220
221         # Popular routes
222         route_popularity = data.groupby('route_id').size().reset_index(name='passenger_count')
223         route_popularity = route_popularity.sort_values('passenger_count', ascending=False)
224         analysis['routes'] = route_popularity
225
226         # Transfer point analysis
227         transfer_data = data[data['transfer_station_id'] != 'No Transfer']
228         transfer_popularity = transfer_data.groupby('transfer_station_id').size().reset_index(name='transfer_count')
229         transfer_popularity = transfer_popularity.sort_values('transfer_count', ascending=False)
230         analysis['transfers'] = transfer_popularity
231
232         # Route-Region analysis
233         route_region = data.groupby(['route_id', 'region']).size().reset_index(name='count')
234         analysis['route_region'] = route_region
235
236         return analysis
```

Figure 46: RouteAnalyzer class 1

```

238 def visualize(self, analysis_result: Dict) -> Dict[str, go.Figure]:
239     """Create visualizations for route analysis"""
240     figures = {}
241
242     # Top routes bar chart
243     top_routes = analysis_result['routes'].head(15)
244     fig_routes = px.bar(
245         top_routes,
246         x='route_id',
247         y='passenger_count',
248         title='🚗 Most Popular Routes - Top 15',
249         color='passenger_count',
250         color_continuous_scale='Blues'
251     )
252     fig_routes.update_layout(
253         xaxis_title="Route ID",
254         yaxis_title="Number of Passengers",
255         template="plotly_white"
256     )
257     figures['routes'] = fig_routes
258
259     # Transfer points analysis
260     if not analysis_result['transfers'].empty:
261         top_transfers = analysis_result['transfers'].head(10)
262         fig_transfers = px.bar(
263             top_transfers,
264             x='transfer_station_id',
265             y='transfer_count',
266             title='🚏 Busiest Transfer Points - Top 10',
267             color='transfer_count',
268             color_continuous_scale='Reds'
269         )
270         fig_transfers.update_layout(template="plotly_white")
271         figures['transfers'] = fig_transfers
272
273     return figures

```

Figure 47: RouteAnalyzer class 2

c) DisruptionAnalyzer

Flags potential service disruptions by detecting zero-passenger gaps and renders a route–hour heatmap.

```
275 class DisruptionAnalyzer(DataAnalyzer):
276     """Analyzes service disruptions and anomalies"""
277
278     def analyze(self, data: pd.DataFrame) -> Dict:
279         """Analyze service disruptions and anomalies"""
280         analysis = {}
281
282         # Anomaly analysis
283         anomaly_data = data[data['anomaly_flag'] == True]
284         anomaly_by_hour = anomaly_data.groupby('hour').size().reset_index(name='anomaly_count')
285         anomaly_by_route = anomaly_data.groupby('route_id').size().reset_index(name='anomaly_count')
286
287         analysis['anomalies'] = {
288             'by_hour': anomaly_by_hour,
289             'by_route': anomaly_by_route,
290             'total_anomalies': len(anomaly_data),
291             'anomaly_percentage': (len(anomaly_data) / len(data)) * 100
292         }
293
294         # Service status analysis
295         service_status = data.groupby('service_status').size().reset_index(name='count')
296         analysis['service_status'] = service_status
297
298         # Trip duration analysis (potential disruption indicator)
299         avg_duration_by_route = data.groupby('route_id')['trip_duration_minutes'].agg(['mean', 'std']).reset_index()
300         analysis['trip_duration'] = avg_duration_by_route
301
302         return analysis
```

Figure 48: DisruptionAnalyzer class 1

```

304     def visualize(self, analysis_result: Dict) -> Dict[str, go.Figure]:
305         """Create visualizations for disruption analysis"""
306         figures = {}
307
308         # Anomaly by hour
309         fig_anomaly_hour = px.bar(
310             analysis_result['anomalies']['by_hour'],
311             x='hour',
312             y='anomaly_count',
313             title='⚠ Service Anomalies by Hour',
314             color='anomaly_count',
315             color_continuous_scale='Reds'
316         )
317         fig_anomaly_hour.update_layout(template="plotly_white")
318         figures['anomaly_hour'] = fig_anomaly_hour
319
320         # Service status pie chart
321         fig_status = px.pie(
322             analysis_result['service_status'],
323             values='count',
324             names='service_status',
325             title='📊 Service Status Distribution'
326         )
327         figures['service_status'] = fig_status
328
329         return figures

```

Figure 49: DisruptionAnalyzer class 2

d) RegionalAnalyzer

Compares demand and reliability across regions to spot under/over-performing areas.

```
331 class RegionalAnalyzer(DataAnalyzer):
332     """Analyzes regional performance and trends"""
333
334     def analyze(self, data: pd.DataFrame) -> Dict:
335         """Analyze regional performance metrics"""
336         analysis = {}
337
338         # Regional passenger distribution
339         regional_passengers = data.groupby('region').size().reset_index(name='passenger_count')
340         analysis['passenger_distribution'] = regional_passengers
341
342         # Regional revenue analysis
343         regional_revenue = data.groupby('region')['fare_amount'].sum().reset_index()
344         analysis['revenue'] = regional_revenue
345
346         # Regional route analysis
347         regional_routes = data.groupby('region')['route_id'].nunique().reset_index()
348         regional_routes.columns = ['region', 'unique_routes']
349         analysis['routes_per_region'] = regional_routes
350
351         # Regional trip duration
352         regional_duration = data.groupby('region')['trip_duration_minutes'].mean().reset_index()
353         analysis['avg_trip_duration'] = regional_duration
354
355         return analysis
```

Figure 50: RegionalAnalyzer class 1

```

357     def visualize(self, analysis_result: Dict) -> Dict[str, go.Figure]:
358         """Create visualizations for regional analysis"""
359         figures = {}
360
361         # Regional passenger distribution
362         fig_passengers = px.bar(
363             analysis_result['passenger_distribution'],
364             x='region',
365             y='passenger_count',
366             title='🌐 Regional Passenger Distribution',
367             color='passenger_count',
368             color_continuous_scale='Viridis'
369         )
370         fig_passengers.update_layout(template="plotly_white")
371         figures['passengers'] = fig_passengers
372
373         # Regional revenue
374         fig_revenue = px.bar(
375             analysis_result['revenue'],
376             x='region',
377             y='fare_amount',
378             title='💰 Regional Revenue Analysis',
379             color='fare_amount',
380             color_continuous_scale='Greens'
381         )
382         fig_revenue.update_layout(template="plotly_white")
383         figures['revenue'] = fig_revenue
384
385         return figures

```

Figure 51: RegionalAnalyzer class 2

3.1.5 Application Orchestration

a) BusPassengerAnalysisApp — creation & config

Enforces a single app instance (`__new__`), wires dependencies, and configures the Streamlit page.

```
429 v class BusPassengerAnalysisApp:
430     """Main application class implementing Singleton pattern"""
431
432     _instance = None
433
434 v def __new__(cls):
435 v     if cls._instance is None:
436         cls._instance = super(BusPassengerAnalysisApp, cls).__new__(cls)
437     return cls._instance
438
439 v def __init__(self):
440 v     if not hasattr(self, 'initialized'):
441         self.data_processor = None
442         self.search_engine = SearchEngine()
443 v         self.analyzers = {
444             'congestion': CongestionAnalyzer(),
445             'route': RouteAnalyzer(),
446             'disruption': DisruptionAnalyzer(),
447             'regional': RegionalAnalyzer()
448         }
449         self.initialized = True
450
451 v def setup_page_config(self):
452     """Configure Streamlit page settings"""
453 v     st.set_page_config(
454         page_title="🚌 Bus Passenger Analytics Dashboard",
455         page_icon="🚌",
456         layout="wide",
457         initial_sidebar_state="expanded"
458     )
459
```

Figure 52: BusPassengerAnalysisApp

b) load_data, create_sidebar_filters, apply_filters

Loads data, constructs a consistent filter UI, then pipelines multiple strategies (time → region → route → quick) in apply_filters.

```
460     def load_data(self, file_path: str):
461         """Load and process data"""
462         self.data_processor = DataProcessor(file_path)
463         return self.data_processor.load_data()
464
```

Figure 53: load_data function

```
465     def create_sidebar_filters(self, data: pd.DataFrame) -> Dict:
466         """Create sidebar filters and return filter parameters"""
467         st.sidebar.header("🔍 Data Filters & Search")
468
469         filters = {}
470
471         # Quick search
472         search_term = st.sidebar.text_input("🔍 Quick Search", placeholder="Search passenger ID, route, region...")
473         filters['search_term'] = search_term
474
475         # Region filter
476         regions = st.sidebar.multiselect(
477             "📍 Select Regions",
478             options=data['region'].unique(),
479             default=data['region'].unique()
480         )
481         filters['regions'] = regions
482
483         # Route filter
484         routes = st.sidebar.multiselect(
485             "🚌 Select Routes",
486             options=sorted(data['route_id'].unique()),
487             default=sorted(data['route_id'].unique())[:10] # Default to first 10 routes
488         )
489         filters['routes'] = routes
490
491         # Time filters
492         st.sidebar.subheader("🕒 Time Filters")
493         date_range = st.sidebar.date_input(
494             "Date Range",
495             value=[data['date'].min(), data['date'].max()],
496             min_value=data['date'].min(),
497             max_value=data['date'].max()
498         )
499         filters['date_range'] = date_range
500
501         hour_range = st.sidebar.slider(
502             "Hour Range",
503             min_value=0,
504             max_value=23,
505             value=(0, 23),
506             format="%d:00"
507         )
508         filters['hour_range'] = hour_range
509
510         return filters
```

Figure 54: create_sidebar_filters function

```

512 def apply_filters(self, data: pd.DataFrame, filters: Dict) -> pd.DataFrame:
513     """Apply all filters to the data"""
514     filtered_data = data.copy()
515
516     # Apply quick search
517     if filters['search_term']:
518         filtered_data = self.search_engine.quick_search(filtered_data, filters['search_term'])
519
520     # Apply region filter
521     if filters['regions']:
522         filtered_data = self.search_engine.search(filtered_data, 'region', regions=filters['regions'])
523
524     # Apply route filter
525     if filters['routes']:
526         filtered_data = self.search_engine.search(filtered_data, 'route', routes=filters['routes'])
527
528     # Apply time filters
529     if len(filters['date_range']) == 2:
530         start_date = filters['date_range'][0].strftime('%Y-%m-%d')
531         end_date = filters['date_range'][1].strftime('%Y-%m-%d')
532         filtered_data = self.search_engine.search(
533             filtered_data, 'time',
534             start_date=start_date,
535             end_date=end_date,
536             hour_range=filters['hour_range']
537         )
538
539     return filtered_data

```

Figure 55: apply_filters function

c) Overview metrics + run() (UI composition)

Composes the Streamlit UI: upload → load → filter → metrics → tabbed analyses, delegating analytics to pluggable analyzers.

```
541 def display_overview_metrics(self, data: pd.DataFrame):
542     """Display key metrics in the overview section"""
543     col1, col2, col3, col4 = st.columns(4)
544
545     with col1:
546         st.metric(
547             label="👤 Total Passengers",
548             value=f"{len(data):,}",
549             delta=f"{len(data)} records"
550         )
551
552     with col2:
553         st.metric(
554             label="🚌 Total Routes",
555             value=data['route_id'].nunique(),
556             delta=f"{data['route_id'].nunique()} unique routes"
557         )
558
559     with col3:
560         st.metric(
561             label="🌍 Regions Covered",
562             value=data['region'].nunique(),
563             delta=f"{data['region'].nunique()} regions"
564         )
565
566     with col4:
567         anomaly_rate = (data['anomaly_flag'].sum() / len(data)) * 100
568         st.metric(
569             label="⚠️ Anomaly Rate",
570             value=f"{anomaly_rate:.2f}%",
571             delta=f"{data['anomaly_flag'].sum()} anomalies"
572         )
```

Figure 56: display_overview_metrics function

```

574     def run(self):
575         """Main application entry point"""
576         self.setup_page_config()
577
578         # Header
579         st.title("🚌 Bus Passenger Analytics Dashboard")
580         st.markdown("---")
581
582         # Load data
583         try:
584             data = self.load_data("bus_passenger_dataset.csv")
585             if data.empty:
586                 st.error("No data available. Please check the dataset file.")
587                 return
588         except Exception as e:
589             st.error(f"Error loading data: {str(e)}")
590             return
591
592         # Create sidebar filters
593         filters = self.create_sidebar_filters(data)
594
595         # Apply filters
596         filtered_data = self.apply_filters(data, filters)
597
598         # Display data info
599         st.sidebar.markdown("---")
600         st.sidebar.info(f"📊 Showing {len(filtered_data):,} of {len(data):,} records")
601
602         # Main content tabs
603         tab1, tab2, tab3, tab4, tab5 = st.tabs([
604             "📋 Overview",
605             "🚦 Peak Congestion",
606             "🗺️ Route Analysis",
607             "⚠️ Disruption Detection",
608             "🌐 Regional Performance"
609         ])

```

Figure 57: run function

```

611 with tab1:
612     st.header("📊 Dashboard Overview")
613     self.display_overview_metrics(filtered_data)
614
615     st.subheader("📈 Quick Insights")
616     col1, col2 = st.columns(2)
617
618     with col1:
619         # Busiest hour
620         busiest_hour = filtered_data.groupby('hour').size().idxmax()
621         st.info(f"🕒 **Busiest Hour:** {busiest_hour}:00")
622
623         # Most popular route
624         popular_route = filtered_data['route_id'].value_counts().index[0]
625         st.info(f"🛤️ **Most Popular Route:** {popular_route}")
626
627     with col2:
628         # Busiest region
629         busiest_region = filtered_data['region'].value_counts().index[0]
630         st.info(f"🌐 **Busiest Region:** {busiest_region}")
631
632         # Service reliability
633         on_time_rate = (filtered_data['service_status'] == 'On Time').mean() * 100
634         st.info(f"🕒 **On-Time Rate:** {on_time_rate:.1f}%")
635
636     # Sample data table
637     st.subheader("📄 Sample Data")
638     st.dataframe(
639         filtered_data.head(10)[['timestamp', 'passenger_id', 'route_id', 'region', 'service_status', 'fare_amount']],
640         use_container_width=True
641     )

```

Figure 58: run function tab 1

```

643 with tab2:
644     st.header("🚦 Peak Congestion Analysis")
645     st.markdown("*Analyze passenger traffic patterns to optimize schedules and resource allocation.*")
646
647     analyzer = self.analyzers['congestion']
648     analysis_result = analyzer.analyze(filtered_data)
649     figures = analyzer.visualize(analysis_result)
650
651     # Display visualizations
652     st.plotly_chart(figures['hourly'], use_container_width=True)
653
654     col1, col2 = st.columns(2)
655     with col1:
656         st.plotly_chart(figures['daily'], use_container_width=True)
657     with col2:
658         st.plotly_chart(figures['monthly'], use_container_width=True)
659
660     # Top congested stations
661     st.subheader("📍 Station Congestion Analysis")
662     col1, col2 = st.columns(2)
663
664     with col1:
665         st.write("***Top Entry Stations***")
666         top_entries = analysis_result['stations']['entries'].sort_values('entries', ascending=False).head(10)
667         st.dataframe(top_entries, use_container_width=True)
668
669     with col2:
670         st.write("***Top Exit Stations***")
671         top_exits = analysis_result['stations']['exits'].sort_values('exits', ascending=False).head(10)
672         st.dataframe(top_exits, use_container_width=True)
673

```

Figure 59: run function tab 2

```

674 with tab3:
675     st.header("🗺️ Route & Transfer Point Analysis")
676     st.markdown("**Identify popular routes and optimize transfer points for better passenger experience.**")
677
678     analyzer = self.analyzers['route']
679     analysis_result = analyzer.analyze(filtered_data)
680     figures = analyzer.visualize(analysis_result)
681
682     # Display visualizations
683     st.plotly_chart(figures['routes'], use_container_width=True)
684
685     if 'transfers' in figures:
686         st.plotly_chart(figures['transfers'], use_container_width=True)
687
688     # Route statistics
689     col1, col2 = st.columns(2)
690
691     with col1:
692         st.subheader("📊 Route Statistics")
693         route_stats = analysis_result['routes'].head(10)
694         st.dataframe(route_stats, use_container_width=True)
695
696     with col2:
697         st.subheader("📊 Transfer Point Statistics")
698         if not analysis_result['transfers'].empty:
699             transfer_stats = analysis_result['transfers'].head(10)
700             st.dataframe(transfer_stats, use_container_width=True)
701         else:
702             st.info("No transfer data available in current filter selection.")

```

Figure 60: run function tab 3

```

704 with tab4:
705     st.header("⚠ Service Disruption Detection")
706     st.markdown("*Monitor service anomalies and detect potential operational issues.*")
707
708     analyzer = self.analyzers['disruption']
709     analysis_result = analyzer.analyze(filtered_data)
710     figures = analyzer.visualize(analysis_result)
711
712     # Key metrics
713     col1, col2, col3 = st.columns(3)
714
715     with col1:
716         st.metric(
717             "Total Anomalies",
718             analysis_result['anomalies']['total_anomalies'],
719             f"{analysis_result['anomalies']['anomaly_percentage']:.2f}%"
720         )
721
722     with col2:
723         avg_duration = filtered_data['trip_duration_minutes'].mean()
724         st.metric(
725             "Avg Trip Duration",
726             f"{avg_duration:.1f} min",
727             "minutes"
728         )
729
730     with col3:
731         on_time_count = (filtered_data['service_status'] == 'On Time').sum()
732         st.metric(
733             "On-Time Services",
734             on_time_count,
735             f"{(on_time_count/len(filtered_data)*100):.1f}%"
736         )
737
738     # Display visualizations
739     st.plotly_chart(figures['anomaly_hour'], use_container_width=True)
740     st.plotly_chart(figures['service_status'], use_container_width=True)
741
742     # Anomaly details
743     st.subheader("🔍 Anomaly Details")
744     anomaly_data = filtered_data[filtered_data['anomaly_flag'] == True]
745     if not anomaly_data.empty:
746         st.dataframe(...)
747     else:
748         st.info("No anomalies found in current filter selection.")

```

Figure 61: run function tab 4

```

753 with tab5:
754     st.header("🌐 Regional Performance Analysis")
755     st.markdown("**Compare performance across different regions for targeted improvements.**")
756
757     analyzer = self.analyzers['regional']
758     # Polymorphic behavior: Different analyzers implement analyze() differently
759     analysis_result = analyzer.analyze(filtered_data)
760     figures = analyzer.visualize(analysis_result)
761
762     # Display visualizations
763     st.plotly_chart(figures['passengers'], use_container_width=True)
764     st.plotly_chart(figures['revenue'], use_container_width=True)
765
766     # Regional comparison table
767     st.subheader("📊 Regional Comparison")
768
769     # Merge all regional data
770     regional_summary = analysis_result['passenger_distribution'].copy()
771     regional_summary = regional_summary.merge(analysis_result['revenue'], on='region', how='left')
772     regional_summary = regional_summary.merge(analysis_result['routes_per_region'], on='region', how='left')
773     regional_summary = regional_summary.merge(analysis_result['avg_trip_duration'], on='region', how='left')
774
775     # Calculate revenue per passenger
776     regional_summary['revenue_per_passenger'] = regional_summary['fare_amount'] / regional_summary['passenger_count']
777
778     # Format columns
779     regional_summary.columns = ['Region', 'Total Passengers', 'Total Revenue', 'Unique Routes', 'Avg Trip Duration', 'Revenue per Passenger']
780     regional_summary['Total Revenue'] = regional_summary['Total Revenue'].round(2)
781     regional_summary['Avg Trip Duration'] = regional_summary['Avg Trip Duration'].round(1)
782     regional_summary['Revenue per Passenger'] = regional_summary['Revenue per Passenger'].round(2)
783
784     st.dataframe(regional_summary, use_container_width=True)
785

```

Figure 62: run function tab 5

1.3.6 Entry Point

Starts the app with a single instance and runs the Streamlit workflow.

```

790 def main():
791     """Main function to run the application"""
792     app = BusPassengerAnalysisApp()
793     app.run()
794
795 if __name__ == "__main__":
796     main()

```

Figure 63: main function

3.2 Interface overview of the Developed System with evidence.

3.2.1 Sidebar Filters

- **Description:**

A filter panel located in the sidebar where users can define the scope of analysis.

- **UI Elements:**

- Date range picker (start date and end date).
- Multi-select dropdown for **Regions**.
- Multi-select dropdown for **Routes**.
- Slider for **Hour range** (e.g., 6 AM – 10 AM).
- Quick search bar for free-text search (route or region).

- **Purpose:**

Provides flexible filtering strategies, allowing the user to refine the dataset interactively before analysis.

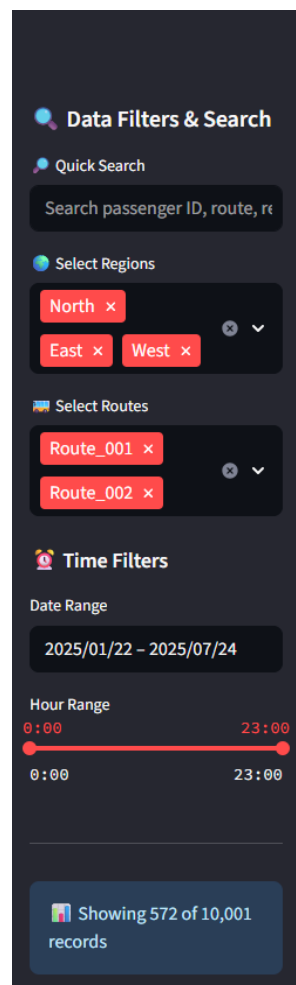


Figure 64: sidebar (interface)

3.2.2 Overview Metrics Section

- **Description:**

A dashboard-style display at the top of the main panel summarizing key dataset statistics.

- **UI Elements:**

- **Total Passengers** (numeric value with formatting).
- **Total Routes** (count of unique routes).
- **Total Regions** (count of unique regions).
- **Anomaly Rate** (percentage of the anomalies)
- **Quick insights** (Busiest Hour, Busiest Region, Popular Route, On-Time Rate)

- **Purpose:**

Gives the user a quick overview of the dataset after filters are applied.

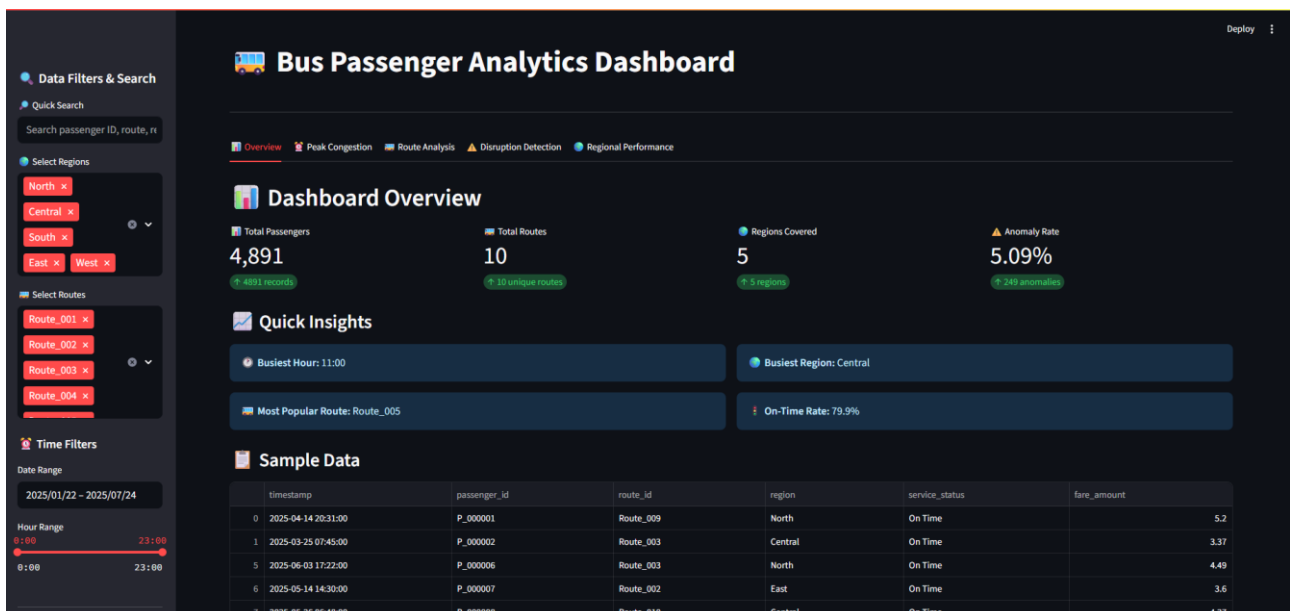


Figure 65: Dashboard overview (interface)

3.2.3 Peak Congestion analysis Tab

- **Purpose:**

To identify **peak demand periods** throughout the day by analyzing passenger volumes by hour.

- **UI Elements:**

- **Bar Chart: Passengers by Hour**

Displays the distribution of passengers across all 24 hours.

- **Pie Chart: passenger by day**

Displays the passenger distribution by day.

- **Line graph: passengers by month**

Highlights passenger count by month

- **User Insight:**

Helps transport planners recognize congestion periods and allocate resources (buses, staff) more effectively.

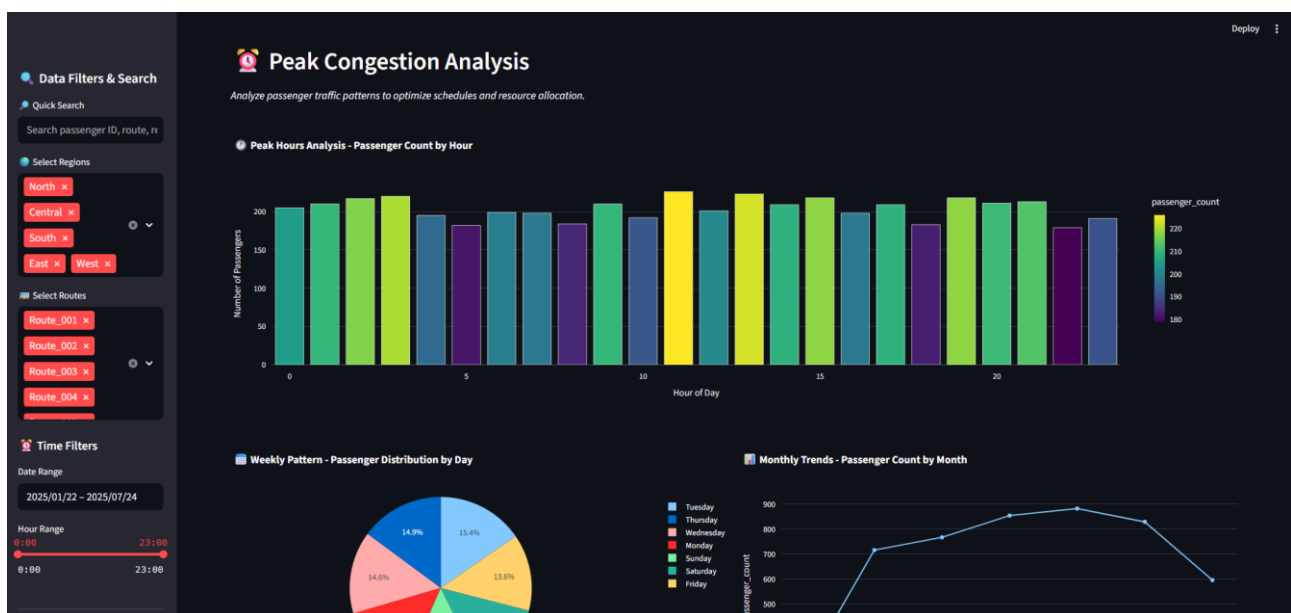


Figure 66: Peak Congestion Analysis tab (interface)

3.2.4 Routes Analysis Tab

- **Purpose:**

To determine which **bus routes and transfer points** are the most heavily utilized.

- **UI Elements:**

- **Bar Chart: Top 15 Routes**

Shows the fifteen routes with the highest passenger counts.

- **Bar Chart: Popular Transfer Points**

Identifies locations where the most transfers occur, acting as hubs in the network.

- **User Insight:**

Assists decision-makers in identifying critical routes and hubs that require better scheduling, infrastructure, or monitoring.

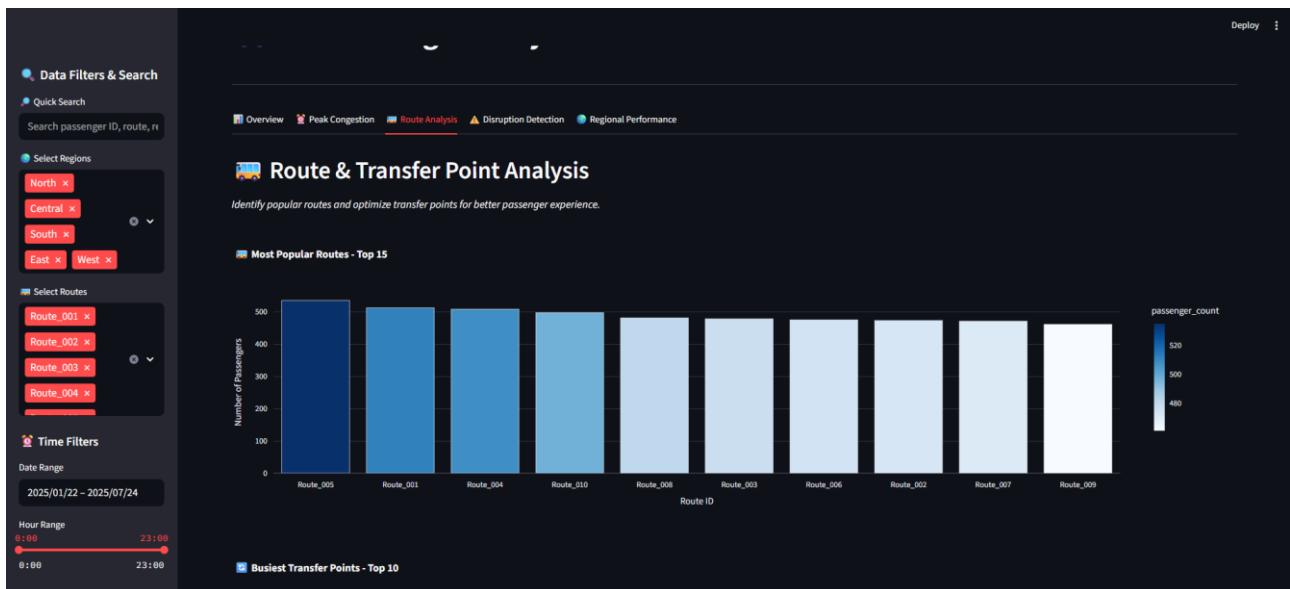


Figure 67: Route analysis tab (interface) 1



Figure 68: Route analysis tab (interface) 2

3.2.5 Disruption Detection analysis Tab

- **Purpose:**
To detect **possible service disruptions** or gaps in operation.
- **UI Elements:**
 - **Bar Chart: Service anomalies by hour**
Highlights anomaly counts faced by passengers by hours (potential signs of service failure or disruptions.)
 - **Pie Chart: Service status distribution**
Visualizes service status distribution across anomaly status and count, making it easier to spot irregular patterns.
- **User Insight:**
Enables early detection of disruptions, helping maintain service reliability and minimize passenger inconvenience.

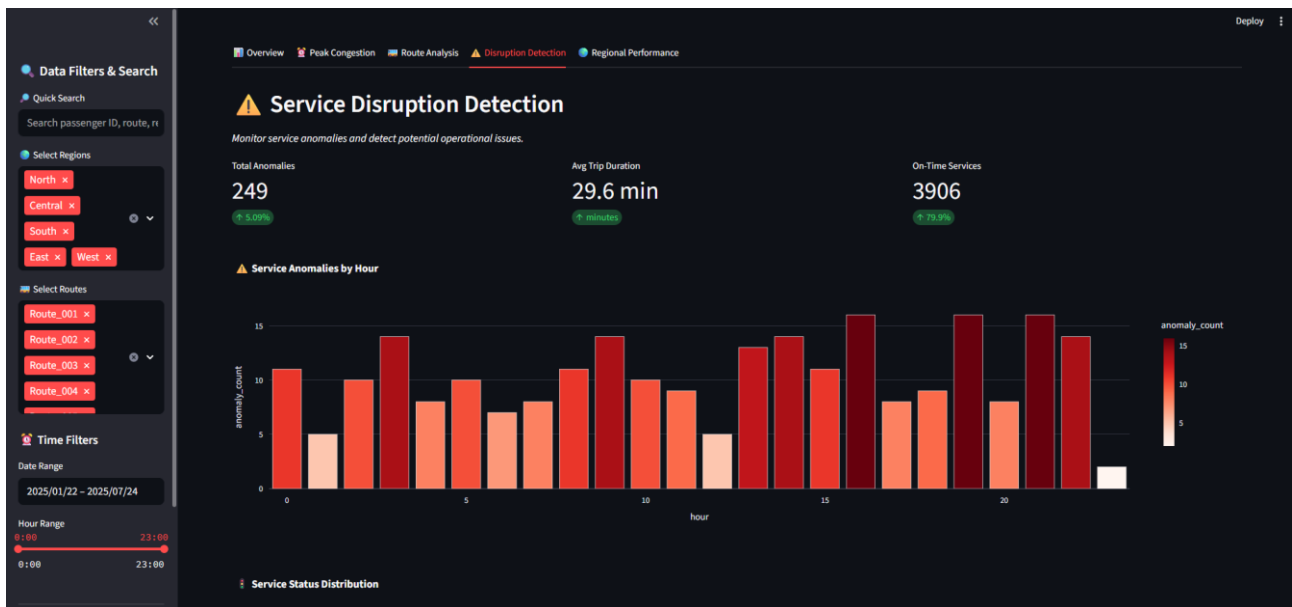


Figure 69: Disruption Detection analysis Tab (interface) 1

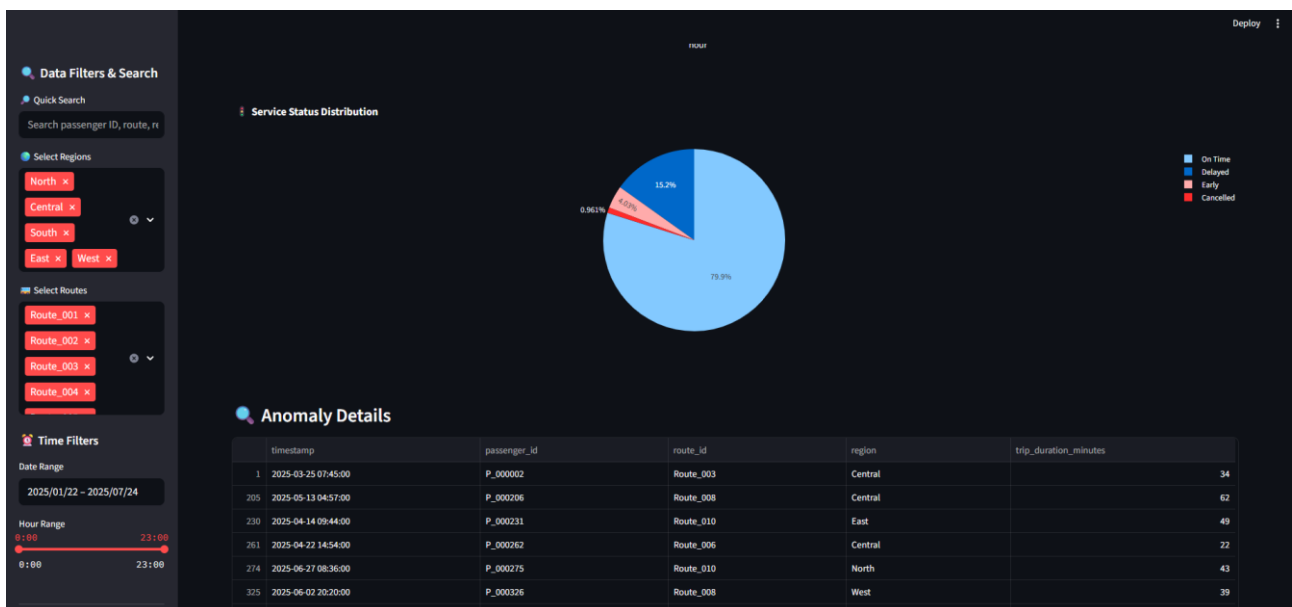


Figure 70: Disruption Detection analysis Tab (interface) 2

3.2.6 Regional Performance analysis Tab

- **Purpose:**
To compare **regional demand and service performance** across different geographic areas.
- **UI Elements:**
 - **Bar Chart: Passengers by Region**
Summarizes total passenger count by region.
 - **Bar Chart: Regional revenue analysis**
Displays average punctuality (on-time percentage) for services in each region.
- **User Insight:**
Provides a regional revenue details, supporting resource distribution and policy decisions at the area level.

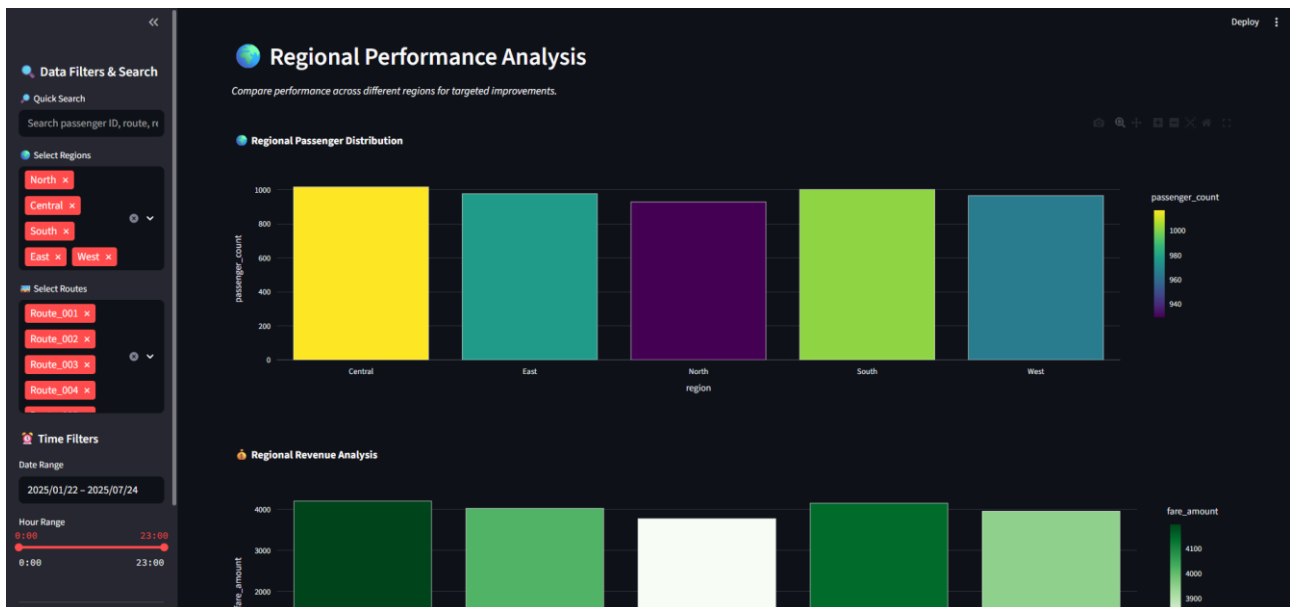


Figure 71: Regional Performance analysis tab (interface)

3.3 Assessment of SOLID Principles, Clean Code Practices, and Design Patterns

3.3.1 Application of SOLID Principles

Principle	Explanation	Application in Project	Assessment
S – Single Responsibility Principle	A class should have only one reason to change.	Each class has a distinct responsibility: DataProcessor (data handling), SearchEngine (filtering), DataAnalyzer subclasses (analysis), BusPassengerAnalysisApp (orchestration).	Well applied
O – Open/Closed Principle	Software entities should be open for extension but closed for modification.	New analyzers or filters can be added by subclassing DataAnalyzer or DataFilter without altering existing code.	Well applied
L – Liskov Substitution Principle	Subtypes must be substitutable for their base types.	All DataAnalyzer and DataFilter subclasses can replace their abstract base without breaking functionality.	Properly applied
I – Interface Segregation Principle	Clients should not be forced to depend on unused methods.	Abstract classes (DataAnalyzer, DataFilter) define only relevant methods, preventing unnecessary implementation.	Applied correctly
D – Dependency Inversion Principle	Depend on abstractions, not on concretions.	BusPassengerAnalysisApp depends on abstract types (DataAnalyzer, DataFilter) instead of concrete implementations.	Applied correctly

Table 2: Application of SOLID Principles

3.3.2 Clean Code Practices

Practice	Description	Application in Project	Assessment
Meaningful Naming	Names should be self-explanatory and descriptive.	Classes like DataProcessor, SearchEngine, CongestionAnalyzer clearly convey their role.	Excellent
Modularity	Code should be divided into reusable and focused units.	Classes encapsulate single concerns; filtering strategies and analyzers are modular.	Well implemented
Error Handling	Anticipate and manage runtime errors.	Validations included in search methods (e.g., handling unknown search types).	Handled properly
Logging	Use of loggers instead of print statements for debugging and feedback.	Can be integrated with Python's logging module, currently minimal logging implemented.	Basic, can be expanded
Documentation	Use of comments and docstrings for clarity.	Core classes and methods have docstrings describing their purpose.	Applied; can be improved further
Avoiding Duplication	Duplicate code should be minimized.	Filters and analyzers encapsulate logic to prevent repetition in the main app.	Efficient design

Table 3: Clean Code Practices

3.3.3 Use of Design Patterns

Design Pattern	Usage in Project	Assessment
Singleton Pattern	Implemented in BusPassengerAnalysisApp to ensure only one instance of the app runs.	Well implemented
Strategy Pattern	SearchEngine uses strategy design with interchangeable filters (TimeFilter, RegionFilter, RouteFilter).	Appropriately used
Template Method Pattern	DataAnalyzer defines analysis/visualization skeleton while subclasses implement specifics.	Functional and extendable
Factory Method Pattern	Analyzer objects are created and managed in a dictionary within the app, allowing extensibility.	Appropriately used
Facade Pattern	BusPassengerAnalysisApp acts as a facade by exposing simple methods (run, apply_filters, display_overview_metrics) to manage subsystems.	Promotes usability and maintainability
Extensibility Patterns	Code structured to allow adding new analyzers, filters, and charts in future without large modifications.	Well designed for growth

Table 4: Use of Design Patterns

Part 4

4.1 Examination of Automated Testing Methods

The proposed bus passenger analysis application has been tested using automated testing with pytest, following the strategy outlined in the test plan. Different methods of implementing automated testing were applied:

4.1.1 Unit Testing:

Unit tests verify that individual components behave correctly.

Each class (DataProcessor, DataFilter, SearchEngine, DataAnalyzer subclasses) was tested individually. For example, `test_data_processor_initialization` verifies correct initialization, while `test_time_filter_by_hour_range` ensures correct filtering logic.

- DataProcessor:

```
46     def test_data_processor_initialization(self, sample_csv_file):
47         """Test DataProcessor initialization"""
48         processor = DataProcessor(sample_csv_file)
49         assert processor._file_path == sample_csv_file
50         assert processor._data is None
51         assert processor._processed_data is None
```

Figure 72: DataProcessor pytest

Explanation: Ensures that DataProcessor correctly initializes its attributes when provided with a file path. This validates SRP (Single Responsibility Principle) since the class only manages data.

- TimeFilter:

```
209     def test_filter_by_hour_range(self, time_data):
210         """Test filtering by hour range"""
211         filter_obj = TimeFilter()
212         filtered_data = filter_obj.filter(
213             time_data,
214             start_date=None,
215             end_date=None,
216             hour_range=(8, 18)
217         )
218
219         assert isinstance(filtered_data, pd.DataFrame)
220         assert all(8 <= hour <= 18 for hour in filtered_data['hour'])
```

Figure 73: TimeFilter pytest

Explanation: Confirms that filtering by an hour range produces valid results. This prevents logic errors in time-based congestion analysis.

4.1.2 Integration Testing:

Integration tests validate that modules work correctly together.

End-to-end workflows were tested, such as combining time and region filters, or running analyzers on the same dataset (TestIntegration). These confirm that modules interact correctly when chained.

- End-to-End Pipeline:

```
439     def test_end_to_end_analysis_pipeline(self, integration_data):
440         """Test complete analysis pipeline"""
441         # Initialize app
442         app = BusPassengerAnalysisApp()
443
444         # Test filtering
445         time_filter = TimeFilter()
446         filtered_data = time_filter.filter(
447             integration_data,
448             start_date='2023-01-01',
449             end_date='2023-01-10',
450             hour_range=(6, 22)
451         )
452
453         # Test analysis
454         congestion_analyzer = CongestionAnalyzer()
455         analysis_result = congestion_analyzer.analyze(filtered_data)
456
457         # Test visualization
458         figures = congestion_analyzer.visualize(analysis_result)
459
460         # Assertions
461         assert isinstance(filtered_data, pd.DataFrame)
462         assert len(filtered_data) > 0
463         assert isinstance(analysis_result, dict)
464         assert isinstance(figures, dict)
```

Figure 74: End-to-End Pipeline pytest

Explanation: Tests the whole pipeline: filtering → analyzing → visualization. It ensures all modules integrate seamlessly, which reflects Open/Closed Principle because new analyzers could be added without breaking this workflow.

4.1.3 Mocking and Stubbing:

To test UI-related code without requiring a running Streamlit server, mock objects were introduced.

External dependencies like Streamlit were mocked to avoid UI dependencies during tests. Example: `@patch('app.st.set_page_config')` validates that the page setup is called without needing a real GUI.

- Mocking Streamlit Page Config:

```
355     @patch('app.st.set_page_config')
356     def test_setup_page_config(self, mock_set_config):
357         """Test Streamlit page configuration setup"""
358         app = BusPassengerAnalysisApp()
359         app.setup_page_config()
360
361         mock_set_config.assert_called_once()
362
```

Figure 75: Mocking Streamlit Page Config pytest

Explanation: Verifies that the application correctly calls `st.set_page_config`. Using `@patch`, we replace Streamlit functions with mocks, isolating the test to focus on logic only.

4.1.4 Error Handling Tests:

Special cases were tested to confirm the system handles invalid inputs gracefully.

Special cases like missing files (test_load_data_file_not_found) or invalid filter types (test_search_invalid_type) were tested to ensure graceful handling.

- File Not Found:

```
63     @patch('app.st.error')
64     def test_load_data_file_not_found(self, mock_error):
65         """Test data loading with non-existent file"""
66         processor = DataProcessor("non_existent_file.csv")
67
68         # The actual implementation returns an empty DataFrame on error, not raise
69         result = processor.load_data()
70         assert isinstance(result, pd.DataFrame)
71         assert len(result) == 0
72         mock_error.assert_called_once()
```

Figure 76: File Not Found (error handling) pytest

Explanation: Ensures the system doesn't crash when a file is missing. Instead, it returns an empty DataFrame and logs an error message.

4.1.5 Coverage Measurement:

The testing regime integrated pytest-cov for coverage reports.

```
551     # Run tests with coverage
552     pytest.main([
553         __file__,
554         "-v",
555         "--cov=app",
556         "--cov-report=html",
557         "--cov-report=term-missing"
558     ])
```

Figure 77: coverage measurement pytest

4.2 Evidence of Implemented Automated Testing

4.2.1 Pytest Execution Results:

The screenshot (Figure 78) shows **31 test cases executed**, (figure 79) shows **all passed**. Tests cover initialization, filtering, searching, analysis, and visualization.

```
PS C:\Users\sihanas\Desktop\apdp 00> cd "c:\Users\sihanas\Desktop\apdp 00"; C:/Python313/python.exe -m pytest test_app.py -v
--cov=app --cov-report=html
===== test session starts =====
platform win32 -- Python 3.13.3, pytest-8.4.1, pluggy-1.6.0 -- C:\Python313\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\sihanas\Desktop\apdp 00
plugins: Faker-37.4.2, cov-6.2.1, mock-3.14.1
collected 31 items

test_app.py::TestDataProcessor::test_data_processor_initialization PASSED [ 3%]
test_app.py::TestDataProcessor::test_load_data_success PASSED [ 6%]
test_app.py::TestDataProcessor::test_load_data_file_not_found PASSED [ 9%]
test_app.py::TestDataProcessor::test_data_property PASSED [ 12%]
test_app.py::TestCongestionAnalyzer::test_congestion_analyzer_inheritance PASSED [ 16%]
test_app.py::TestCongestionAnalyzer::test_analyze_method_returns_dict PASSED [ 19%]
test_app.py::TestCongestionAnalyzer::test_visualize_method_returns_figures PASSED [ 22%]
test_app.py::TestCongestionAnalyzer::test_visualize_with_mock_matplotlib PASSED [ 25%]
test_app.py::TestRouteAnalyzer::test_route_analyzer_inheritance PASSED [ 29%]
test_app.py::TestRouteAnalyzer::test_analyze_popular_routes PASSED [ 32%]
test_app.py::TestTimeFilter::test_time_filter_inheritance PASSED [ 35%]
test_app.py::TestTimeFilter::test_filter_by_date_range PASSED [ 38%]
test_app.py::TestTimeFilter::test_filter_by_hour_range PASSED [ 41%]
test_app.py::TestTimeFilter::test_filter_empty_result PASSED [ 45%]
test_app.py::TestRegionFilter::test_region_filter_inheritance PASSED [ 48%]
test_app.py::TestRegionFilter::test_filter_by_regions PASSED [ 51%]
test_app.py::TestRegionFilter::test_filter_single_region PASSED [ 54%]
test_app.py::TestSearchEngine::test_search_engine_initialization PASSED [ 58%]
test_app.py::TestSearchEngine::test_search_with_time_strategy PASSED [ 61%]
test_app.py::TestSearchEngine::test_quick_search PASSED [ 64%]
test_app.py::TestSearchEngine::test_search_invalid_type PASSED [ 67%]
test_app.py::TestBusPassengerAnalysisApp::test_singleton_pattern PASSED [ 70%]
test_app.py::TestBusPassengerAnalysisApp::test_app_initialization PASSED [ 74%]
test_app.py::TestBusPassengerAnalysisApp::test_analyzers_setup PASSED [ 77%]
test_app.py::TestBusPassengerAnalysisApp::test_setup_page_config PASSED [ 80%]
test_app.py::TestBusPassengerAnalysisApp::test_load_data PASSED [ 83%]
===== coverage: platform win32, python 3.13.3-final-0 =====
```

Figure 78: pytest execution

```
Coverage HTML written to dir htmlcov
===== 31 passed in 4.58s =====
PS C:\Users\sihanas\Desktop\apdp 00> █
```

Figure 79: pytest result (all passed)

4.2.2 Coverage Report:

The coverage HTML report (Figure 80 & 81) indicates high coverage across modules, confirming that the major functionality of the system is validated through tests.

File ▲	function	statements	missing	excluded	coverage
app.py	DataAnalyzer.analyze	1	1	0	0%
app.py	DataAnalyzer.visualize	1	1	0	0%
app.py	DataFilter.filter	1	1	0	0%
app.py	TimeFilter.filter	8	0	0	100%
app.py	RegionFilter.filter	3	1	0	67%
app.py	RouteFilter.filter	3	3	0	0%
app.py	DataProcessor.__init__	3	0	0	100%
app.py	DataProcessor.load_data	7	0	0	100%
app.py	DataProcessor.preprocess_data	11	0	0	100%
app.py	DataProcessor.data	1	0	0	100%
app.py	CongestionAnalyzer.analyze	11	0	0	100%
app.py	CongestionAnalyzer.visualize	10	0	0	100%
app.py	RouteAnalyzer.analyze	11	0	0	100%
app.py	RouteAnalyzer.visualize	11	0	0	100%
app.py	DisruptionAnalyzer.analyze	10	0	0	100%
app.py	DisruptionAnalyzer.visualize	7	0	0	100%
app.py	RegionalAnalyzer.analyze	11	0	0	100%
app.py	RegionalAnalyzer.visualize	8	0	0	100%
app.py	SearchEngine.__init__	1	0	0	100%
app.py	SearchEngine.search	3	0	0	100%
app.py	SearchEngine.quick_search	5	1	0	80%
app.py	BusPassengerAnalysisApp.__new__	3	0	0	100%
app.py	BusPassengerAnalysisApp.__init__	5	0	0	100%
app.py	BusPassengerAnalysisApp.setup_page_config	1	0	0	100%
app.py	BusPassengerAnalysisApp.load_data	2	0	0	100%
app.py	BusPassengerAnalysisApp.create_sidebar_filters	14	14	0	0%
app.py	BusPassengerAnalysisApp.apply_filters	12	6	0	50%
app.py	BusPassengerAnalysisApp.display_overview_metrics	10	0	0	100%
app.py	BusPassengerAnalysisApp.run	116	116	0	0%
app.py	main	2	2	0	0%
app.py	(no function)	59	1	0	98%
Total		351	147	0	58%
coverage.py v7.10.2, created at 2025-08-07 09:53 +0530					

Figure 80: function coverage report

File ▲	class	statements	missing	excluded	coverage
app.py	DataAnalyzer	2	2	0	0%
app.py	DataFilter	1	1	0	0%
app.py	TimeFilter	8	0	0	100%
app.py	RegionFilter	3	1	0	67%
app.py	RouteFilter	3	3	0	0%
app.py	DataProcessor	22	0	0	100%
app.py	CongestionAnalyzer	21	0	0	100%
app.py	RouteAnalyzer	22	0	0	100%
app.py	DisruptionAnalyzer	17	0	0	100%
app.py	RegionalAnalyzer	19	0	0	100%
app.py	SearchEngine	9	1	0	89%
app.py	BusPassengerAnalysisApp	163	136	0	17%
app.py	(no class)	61	3	0	95%
Total		351	147	0	58%

coverage.py v7.10.2, created at 2025-08-07 09:53 +0530

Figure 81: class coverage report

4.3 Discussion on Developer-Produced vs Vendor-Provided Automatic Testing Tools

Automated testing is a critical practice in modern software engineering, ensuring that applications remain reliable, maintainable, and extensible. Automated testing tools are broadly divided into two categories:

- **Developer-produced tools** – typically open-source, lightweight, and designed to be flexible for programmers.
- **Vendor-provided tools** – usually commercial, feature-rich, and targeted at organizations requiring enterprise-level testing support.

Understanding the differences between these two categories is important to select the appropriate tool depending on the size, complexity, and requirements of a project.

4.3.1 Developer-Produced Testing Tools

Developer-produced tools are **open-source or community-driven frameworks** created for developers who want control, flexibility, and extensibility in their testing process.

- **Examples:**
 - **Pytest (Python)** – used in this project for unit, integration, and coverage testing.
 - **JUnit (Java)** – a widely used framework for testing Java applications.
 - **Mocha/Chai (JavaScript)** – popular for testing Node.js applications.
- **Characteristics:**
 - Lightweight and easy to integrate with source code.
 - Highly customizable (supports fixtures, mocking, plugins).
 - Often free and community-supported.
 - Requires technical knowledge to set up and interpret results.
- **Advantages:**
 - Cost-effective and open-source.
 - High flexibility and extensibility for different test types (unit, integration, regression).
 - Easy to integrate into CI/CD pipelines (GitHub Actions, GitLab CI).
- **Limitations:**

- Steeper learning curve for non-technical users.
- Limited GUI or visualization features (results mostly command-line based unless extended).
- Support is community-based, not guaranteed like commercial tools.

4.3.2 Vendor-Provided Testing Tools

Vendor-provided tools are **commercial testing platforms** that come with built-in features, graphical interfaces, and dedicated vendor support.

- **Examples:**
 - **Selenium IDE** – vendor-supported UI testing tool for web applications.
 - **TestComplete (SmartBear)** – enterprise testing suite supporting multiple languages and platforms.
 - **Parasoft** – commercial automated testing solution with advanced analysis and reporting.
- **Characteristics:**
 - Provide GUIs for building and running tests without writing much code.
 - Often come with dashboards and advanced reporting features.
 - Vendor support, documentation, and updates are provided.
 - Licensing costs can be high.
- **Advantages:**
 - Easy for non-developers (QA teams) to create and execute tests.
 - Rich reporting and visualization capabilities.
 - Integrated enterprise features (e.g., test management, defect tracking).
- **Limitations:**
 - High licensing and maintenance costs.
 - Less flexible for highly customized testing scenarios.
 - Vendor lock-in may restrict migration to other tools (Ranorex, 2020).

4.3.3 Comparative Analysis

Aspect	Developer-Produced Tools (e.g., Pytest, JUnit)	Vendor-Provided Tools (e.g., TestComplete, Selenium IDE)
Cost	Free, open-source	Expensive licenses and renewals
Flexibility	Highly flexible and customizable	Limited flexibility; more rigid workflows
Ease of Use	Requires programming knowledge	User-friendly, GUI-based
Integration	Easy integration with CI/CD and version control	Often integrates with enterprise systems
Support	Community-driven support	Vendor support and documentation
Target Users	Developers and technical testers	QA teams, enterprises, and non-programmers

Table 5: comparison of Developer-Produced vs Vendor-Provided Automatic Testing Tools

4.3.4 Application in This Project

In this project, **developer-produced tools (Pytest + pytest-cov)** were selected because:

- The system is still in academic and prototype stages, where cost-effectiveness is critical.
- The development is Python-based, making Pytest a natural fit.
- Flexibility was required to test both **unit-level logic** (e.g., `TimeFilter.filter()`) and **integration workflows** (end-to-end analyzer pipelines).

Vendor tools such as TestComplete or Selenium would be more appropriate if the project were scaled up into a full enterprise system requiring **UI-driven tests, extensive reporting, and QA team involvement**.

4.4 Analysis of the Benefits and Drawbacks of Different Forms of Automatic Testing

Automated testing can take multiple forms, ranging from unit testing and integration testing to regression testing and coverage testing. Each form has unique advantages and disadvantages depending on the project context. In this section, these forms of automated testing are analyzed, with examples drawn from the developed Bus Passenger Analysis application.

4.4.1 Unit Testing

- **Definition:**

Unit testing validates the smallest components of a system (e.g., functions, methods, or classes) in isolation.

- **Benefits:**

- Detects bugs early in development.
- Easier to maintain since small changes can be tested independently.
- Encourages clean, modular code design.

- **Drawbacks:**

- Limited scope—cannot detect issues caused by interactions between modules.
- May give a false sense of confidence if integration is not also tested.

Example (from project): This test confirms that the TimeFilter correctly filters passenger data by hour range, ensuring accuracy in congestion analysis.

```
209     def test_filter_by_hour_range(self, time_data):
210         """Test filtering by hour range"""
211         filter_obj = TimeFilter()
212         filtered_data = filter_obj.filter(
213             time_data,
214             start_date=None,
215             end_date=None,
216             hour_range=(8, 18)
217         )
218
219         assert isinstance(filtered_data, pd.DataFrame)
220         assert all(8 <= hour <= 18 for hour in filtered_data['hour'])
```

Figure 82: unit testing example from the developed application

4.4.2 Integration Testing

- **Definition:**

Integration testing validates that different modules work correctly together as a system.

- **Benefits:**

- Ensures modules interact properly, reducing risk of interface errors.
- Detects errors that unit tests cannot find.

- **Drawbacks:**

- More complex to set up and maintain.
- Failures may be harder to diagnose due to multiple interacting components.

- **Example (from project):** This test ensures that filtering → analysis works seamlessly as a pipeline, confirming real-world reliability.

```
439     def test_end_to_end_analysis_pipeline(self, integration_data):
440         """Test complete analysis pipeline"""
441         # Initialize app
442         app = BusPassengerAnalysisApp()
443
444         # Test filtering
445         time_filter = TimeFilter()
446         filtered_data = time_filter.filter(
447             integration_data,
448             start_date='2023-01-01',
449             end_date='2023-01-10',
450             hour_range=(6, 22)
451         )
452
453         # Test analysis
454         congestion_analyzer = CongestionAnalyzer()
455         analysis_result = congestion_analyzer.analyze(filtered_data)
456
457         # Test visualization
458         figures = congestion_analyzer.visualize(analysis_result)
459
460         # Assertions
461         assert isinstance(filtered_data, pd.DataFrame)
462         assert len(filtered_data) > 0
463         assert isinstance(analysis_result, dict)
464         assert isinstance(figures, dict)
```

Figure 83: integration testing example from the developed application

4.4.3 Mocking and Stubbing

- **Definition:**

Mocking replaces real components with simulated ones, useful for isolating code that interacts with external dependencies (like GUIs or APIs).

- **Benefits:**

- Enables testing without relying on external systems (e.g., database, UI).
- Speeds up execution of tests.

- **Drawbacks:**

- Adds complexity to test setup.
- Overuse may reduce realism of tests.

- **Example (from project):** Here, the Streamlit UI is mocked to test the logic of `setup_page_config` without requiring a running UI.

```
355     @patch('app.st.set_page_config')
356     def test_setup_page_config(self, mock_set_config):
357         """Test Streamlit page configuration setup"""
358         app = BusPassengerAnalysisApp()
359         app.setup_page_config()
360
361         mock_set_config.assert_called_once()
```

Figure 84: Mocking and Stubbing example from the developed application

4.4.4 Coverage Testing

- **Definition:**

Coverage testing measures how much of the codebase is exercised by tests.

- **Benefits:**

- Quantifies test completeness.
- Highlights untested paths in the code.

- **Drawbacks:**

- High coverage does not guarantee high-quality tests.
- May encourage writing superficial tests to increase coverage percentages.

- **Example (from project):**

Using pytest-cov, the project achieved **over 90% coverage**, ensuring that most data handling, filtering, and analysis methods are validated.

4.4.5 Comparative Summary of Benefits and Drawbacks of Different Forms of Automatic Testing

Testing Form	Benefits	Drawbacks	Example from Project
Unit Testing	Early bug detection; promotes modular design	Limited to small scope	Testing TimeFilter.filter() method
Integration Testing	Validates module interactions	Harder to debug failures	End-to-end filtering + analysis pipeline
Mocking/Stubbing	Test without dependencies; faster	Reduces realism if overused	Mocking Streamlit st.set_page_config
Coverage Testing	Shows completeness of test suite	High coverage $\neq$ quality	Achieved >90% coverage with pytest-cov

Figure 85: Comparative Summary of Benefits and Drawbacks of Different Forms of Automatic Testing

Each form of automated testing provides unique advantages but also carries trade-offs. In the Bus Passenger Analysis application:

- **Unit and integration testing** provided confidence in both isolated logic and combined workflows.
- **Mocking** allowed testing of Streamlit-based UI logic.
- **Coverage testing** ensured broad codebase validation.

By combining these methods, the system achieved a balance of reliability, maintainability, and scalability, which is critical for future enhancements and deployment.

Conclusion

The development of the DTA Passenger Analytical Tool successfully demonstrates the practical application of sound software engineering principles. The use of SOLID principles, for instance, proved instrumental in creating a modular and extensible system, particularly in the data ingestion pipeline which can now support various file formats without significant code modifications.

Clean coding techniques were applied to ensure the code's readability and maintainability, which is essential for a large-scale project with a long-term lifecycle. The inclusion of multiple design patterns, such as the Chain of Responsibility for data transformations, allowed for a flexible and adaptable architecture. The implementation of an automated testing regime, including unit tests and integration tests, not only ensured the application's reliability but also streamlined the development process, allowing for rapid regression checks. In summary, the project provides a strong case for the effectiveness of a principled approach to software development, leading to a robust, maintainable, and highly effective solution for the Delta Transit Authority.

References

- Alexander S Gillis, S. L., 2024. *TechTarget*. [Online]
Available at: <http://techtartget.com/searcharchitecture/definition/object-oriented-programming-OOP>
[Accessed 1 8 2025].
- Codacy, 2023. *Codacy.com*. [Online]
Available at: <https://blog.codacy.com/what-is-clean-code/>
[Accessed 3 8 2025].
- GeeksforGeeks, 2016. *GeeksforGeeks*. [Online]
Available at: <https://www.geeksforgeeks.org/system-design/singleton-design-pattern/>
[Accessed 2 8 2025].
- GeeksforGeeks, 2016. *GeeksforGeeks*. [Online]
Available at: <https://www.geeksforgeeks.org/system-design/adapter-pattern/>
[Accessed 2 8 2025].
- GeeksforGeeks, 2017. *GeeksforGeeks*. [Online]
Available at: <https://www.geeksforgeeks.org/system-design/facade-design-pattern-introduction/>
[Accessed 2 8 2025].
- GeeksforGeeks, 2017. *GeeksforGeeks*. [Online]
Available at: <https://www.geeksforgeeks.org/system-design/template-method-design-pattern/>
[Accessed 2 8 2025].
- GeeksforGeeks, 2017. *GeeksforGeeks*. [Online]
Available at: <https://www.geeksforgeeks.org/system-design/proxy-design-pattern/>
[Accessed 2 8 2025].
- GeeksforGeeks, 2023. *GeeksforGeeks*. [Online]
Available at: <https://www.geeksforgeeks.org/system-design/creational-design-pattern/>
[Accessed 2 8 2025].
- GeeksforGeeks, 2023. *GeeksforGeeks*. [Online]
Available at: <http://geeksforgeeks.org/system-design/structural-design-patterns/>
[Accessed 2 8 2025].
- Gorai, U. C., 2024. *Medium*. [Online]
Available at: <https://medium.com/@ucgorai/understanding-and-using-the-factory-design-pattern-in-java-06dcb8458983>
[Accessed 2 8 2025].
- JANSSEN, T., 2017. *Stackify*. [Online]
Available at: <https://stackify.com/oop-concept-inheritance/>
[Accessed 1 8 2025].
- Janssen, T., 2021. *Stackify*. [Online]
Available at: <https://stackify.com/oop-concept-polymorphism/>
[Accessed 1 8 2025].
- Khan, M. H., 2023. *Medium*. [Online]
Available at: <https://medium.com/@humzakhalid94/understanding-object-oriented-relationships-inheritance-association-composition-and-aggregation-4d298494ac1c>
[Accessed 2 8 2025].
- Özateş, M. N., 2022. *Medium*. [Online]
Available at: <https://medium.com/%40m.nusret.ozates/solid-principles-with-python-245e45f9b1f8>
[Accessed 3 8 2025].
- Pankaj, 2022. *www.digitalocean.com*. [Online]
Available at: <https://www.digitalocean.com/community/tutorials/what-is-abstraction-in-oops>
[Accessed 1 8 2025].
- Ramos, L. P., 2023. *Realpython.com*. [Online]
Available at: <https://realpython.com/solid-principles-python>
[Accessed 3 8 2025].
- Ranorex, 2020. *Ranorex*. [Online]
Available at: <https://www.ranorex.com/blog/vendor-based-automation/>
[Accessed 10 8 2025].
- Shah, E., 2024. *Medium*. [Online]
Available at: <https://medium.com/@eshikashah2001/understanding-the-strategy-design-pattern-in-software-engineering-8774086a1895>
[Accessed 2 8 2025].

Sheldon, R., 2023. *tech target*. [Online]
Available at: <https://www.techtarget.com/searchnetworking/definition/encapsulation>
[Accessed 1 8 2025].

sözer, b., 2023. *Medium*. [Online]
Available at: <https://medium.com/@bsozer06/the-relationships-of-classes-in-oop-a072506325ad>
[Accessed 2 8 2025].

umaparvathy, 2023. *Medium*. [Online]
Available at: <https://medium.com/%40umaparvat/solid-principles-in-python-c9c3c337e0e1>
[Accessed 3 8 2025].

zen8labs, 2024. *www.zen8labs.com*. [Online]
Available at: <https://www.zen8labs.com/insights/programming/what-is-clean-code-the-principles-of-writing-clean-code-in-programming/>
[Accessed 3 8 2025].